



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

ФАКУЛТЕТ КОМПЮТЪРНИ СИСТЕМИ И ТЕХНОЛОГИИ

КУРСОВ ПРОЕКТ ЗА ОСВОБОЖДАВАНЕ

Дисциплина: „Програмиране за мобилни устройства”

**Тема: CalorieCalculator - Мобилно приложение за
проследяване на калории**

Изготвили:

1. Мартин Георгиев, Фак. № 121223153, група 38
2. Красимир Гушмаков, Фак. № 121223008, група 38
3. Маргарита Баракова, Фак. № 121223007, група 38

София, 2026

Съдържание

1. Увод.....	4
1.1. Цели на проекта.....	4
1.2. Използвани технологии.....	5
2. Анализ на проблемната област.....	6
2.1. Идентифициране на потребностите.....	6
2.2. Предимства на предложеното решение.....	7
3. Проектиране.....	7
3.1. Целева аудитория.....	7
3.2. Архитектура на системата.....	8
3.2.1. Клиентски слой (Presentation Layer).....	8
3.2.2. Сървърен слой (Business Logic Layer).....	8
3.2.3. Слой за данни (DataLayer).....	9
3.3. Архитектурна диаграма.....	9
3.4. Модел на данните.....	10
3.4.1. Връзки между таблиците.....	11
3.5. Процеси в системата.....	13
3.6. Проектиране на потребителски интерфейс.....	15
3.6.1. Навигационна карта.....	15
3.7. Формули за изчисляване на калории.....	16
3.7.1. Базален метаболизъм (BMR).....	16
3.7.2. Дневен енергиен разход (TDEE).....	16
3.7.3. Корекция спрямо целта.....	17
3.7.4. Разпределение на макронутриентите.....	17
4. Реализация.....	18
4.1. Структура на проекта.....	18
4.1.1. CalorieCalculator (MAUI).....	18
4.1.2. CalorieCalculator.API.....	18
4.1.3. Services.....	19
4.1.4. DataLayer.....	20
4.2. Реализация на автентикация (Красимир).....	20
4.2.1. ISO/IEC 27001 валидация на пароли.....	21
4.2.2. Забравена парола – тристъпков wizard.....	21
4.2.3. Двухфакторна биометрична автентикация с пръстов отпечатък.....	22
4.3. Реализация на лични настройки и статистика (Маргарита).....	23
4.3.1. Профилна снимка чрез селфи камерата.....	23
4.3.2. Редактиране на профилни детайли.....	23
4.3.3. Калорийни пръстени в календара.....	24
4.4. Реализация на калорийни планове (Мартин).....	24
4.4.1. Създаване и автоматично попълване.....	24
4.4.2. Структура на ястията.....	24
4.4.3. Навигация между планове.....	25

4.5. CRUD операции върху храни в ястие (MealFoods) (Мартин).....	25
4.5.1. Добавяне.....	25
4.5.2. Преглед.....	25
4.5.3. Редактиране на грамаж.....	25
4.5.4. Изтриване.....	25
4.6. CRUD операции върху хранителни продукти (FoodProducts) (Мартин).....	26
4.6.1. Създаване и редактиране.....	26
4.6.2. Преглед.....	26
4.7. Баркод скенер (Мартин).....	26
4.8. Мрежова свързаност (Wi-Fi / мобилни данни).....	27
4.9. Модели на данните (Entities).....	28
4.10. Външни API endpoint-и.....	32
4.11. Реализация на Repository Pattern.....	35
4.12. Системи за миграции.....	35
4.13. Калориен календар.....	36
5. Потребителско ръководство.....	36
5.1. Регистрация и вход.....	36
5.2. Забравена парола.....	38
5.3. Настройка на профила.....	39
5.4. Главна страница.....	42
5.5. Добавяне на храна.....	43
5.6. Управление на ястие.....	44
5.7. Създаване на нов продукт.....	44
5.8. Сканиране на баркод.....	45
5.9. Калориен календар.....	46
5.10. Профилна страница.....	48
5.11. Биометричен вход.....	49
5.12. Прогрес.....	51
6. Заключение.....	52
6.1. Постигнати резултати.....	52
6.2. Бъдещо развитие.....	52
7. Приложение – ключови фрагменти от кода.....	53
7.1. IRepository интерфейс.....	53
7.2. IAuthorization интерфейс.....	54
7.3. MigrationRunner.....	54
7.4. Пример за SQL миграция.....	56
7.5. ApiServer (MAUI).....	57
8. Литература + линк за GitLab.....	58

1. Увод

В съвременния свят здравословното хранене и поддържането на оптимално тегло са сред основните приоритети на все повече хора. Въпреки нарастващото осъзнаване на значението на балансираната диета, мнозина срещат затруднения при проследяването на ежедневния си калориен прием и разпределението на макронутриентите. Липсата на удобен и интуитивен инструмент за тази цел води до непоследователност в хранителните навици и трудности при постигането на поставените здравословни цели.

Настоящият проект представя CalorieCalculator – мобилно приложение за платформа Android, разработено с технологията .NET MAUI. Приложението предоставя на потребителите възможност да проследяват ежедневния си калориен прием, да управляват хранителните си планове и да анализират напредъка си чрез интуитивен потребителски интерфейс.

Основната цел на проекта е да създаде цялостно решение, което обхваща пълния цикъл на проследяване на калории – от регистрацията на потребителя, през създаването на персонализиран хранителен план, до визуализацията на напредъка във времето. Приложението използва научно обосновани формули за изчисляване на дневния калориен прием, базирани на индивидуалните характеристики на всеки потребител.

Проектът е разработен от екип от трима студенти в рамките на университетски курс, като целта е не само да се създаде функционален продукт, но и да се приложат съвременни архитектурни подходи и добри практики в софтуерното инженерство.

1.1. Цели на проекта

Основните цели, които проектът си поставя, са следните:

- Разработка на мобилно приложение за Android с .NET MAUI, което предоставя удобен интерфейс за проследяване на калориен прием.
- Изграждане на сървърна архитектура с ASP.NET Core Web API, осигуряваща сигурна и мащабируема комуникация между клиента и базата данни.
- Използване на PostgreSQL база данни, хоствана в облака чрез Aiven, за съхранение на потребителски данни, хранителни продукти и калорийни планове.

- Прилагане на съвременни архитектурни шаблони като Repository Pattern, CQRS с MediatR и слоеста архитектура.
- Използване на четири хардуерни функции на мобилното устройство: камера за сканиране на баркодове (ZXing.Net.Maui), камера за заснемане на профилна снимка/селфи (MAUI MediaPicker), биометричен сензор за пръстов отпечатък като двуфакторна автентикация (Plugin.Fingerprint) и Wi-Fi/мобилни данни за мрежова комуникация с API сървър.
- Осигуряване на екипна работа чрез GitLab с GitFlow стратегия за управление на версиите.

1.2. Използвани технологии

За реализацията на проекта са използвани следните технологии и инструменти:

Технология	Предназначение
.NET MAUI	Разработка на мобилното приложение за Android
ASP.NET Core Web API	Сървърна част - REST API за обработка на заявки
PostgreSQL	Релационна база данни за съхранение на всички данни
Aiven	Облачен хостинг за PostgreSQL базата данни
Entity Framework Core	ORM за достъп до базата данни от .NET
MediatR	Реализация на CQRS шаблона за разделяне на четене и писане
ZXing.Net.Maui	Сканиране на баркодове чрез камерата на устройството
MAUI MediaPacker	Вграден API на .NET MAUI за заснемане на профилна снимка (селфи) чрез камерата на устройството
Plugin.Fingerprint	Двуфакторна биометрична автентикация с пръстов отпечатък (Android)
CommunityToolkit.Maui	Допълнителни MAUI контроли, behaviors и IconTintColorBehavior за навигацията
Open Food Facts API	Външен API за автоматично извличане на хранителна информация по баркод

GitLab	Система за контрол на версиите и екипно сътрудничество
Visual Studio 2022	Интегрирана среда за разработка (IDE)

Таблица 1 - Всяка използвана технология и предназначението ѝ в проекта

2. Анализ на проблемната област

Проследяването на калориен прием е процес, който изисква систематичност, достъп до актуална хранителна информация и удобен начин за въвеждане на данни. Традиционните методи като хартиени дневници, електронни таблици или ръчно изчисляване са неудобни, неефективни, отнемат много време и са податливи на грешки.

Съвременните мобилни технологии предоставят възможност за автоматизиране на този процес чрез използването на хардуерните възможности на мобилните ни устройства: камера за сканиране на баркодовете на хранителните продукти, мрежова свързаност за синхронизация с облачни бази данни и сензори и методи за проследяване на дневния прием на калории на потребителя.

2.1. Идентифициране на потребностите

При анализа на проблемната област бяха идентифицирани следните основни потребности на целевите потребители:

- Бързо и лесно въвеждане на консумирани храни с минимален брой стъпки.
- Автоматично изчисляване на хранителните стойности на базата на въведения грамаж.
- Персонализиран калориен план, съобразен с индивидуалните характеристики и цели, които потребителя си е поставил.
- Възможност за сканиране на баркод за бързо и точно добавяне на продукти.
- Визуализация на напредъка и историята на хранене и ежедневния прием на калории от момента на създаване на профила до текущия ден.
- Осигуряване на сигурност на личните данни и паролата на потребителя чрез биометричен вход и двуфакторна автентикация.

2.2. Предимства на предложеното решение

CalorieCalculator се отличава от конвенционалните подходи за проследяване на калории по следните начини:

1. Използване на научнообосновани формули (Mifflin-St Jeor) за изчисляване на базалния метаболизъм и дневните калорийни нужди на потребителя.
2. Интеграция с Open Food Facts API за автоматично попълване на хранителна информация при сканиране на баркод.
3. Облачна архитектура, позволяваща достъп до данните от различни устройства - смартфон, лаптоп, компютър.
4. Собствена система за миграции на базата данни, осигуряваща версионен контрол на схемата.
5. Калориен календар за преглед на историческите данни по месеци от момента на създаване на профила до днешния ден. Чрез него потребителя може асинхронно да вижда какъв процент от дневния си лимит на калориите е приел, да вижда дните, в които е прекрачил дневния си лимит, и дните, в които е успял да влезе в лимитът си и се справил според изготвеният му план.

3. Проектиране

3.1. Целева аудитория

Приложението CalorieCalculator е предназначено за следните групи потребители:

- Хора, които желаят да отслабнат чрез контролиран калориен дефицит.
- Спортисти и фитнес ентусиасти, които проследяват макронутриентите си.
- Хора с определени хранителни цели – набиране на мускулна маса, поддържане на тегло.
- Всеки, който желае да има по-добра представа за хранителния си режим.

3.2. Архитектура на системата

Системата е изградена на принципа на тристепенната архитектура (three-tier architecture), като всеки слой има ясно дефинирана отговорност. Решението е организирано в четири проекта в рамките на едно Visual Studio решение (Solution), които комуникират помежду си чрез ясно дефинирани интерфейси и Dependency Injection. Използваните архитектурни шаблони включват: MVVM (Model-View-ViewModel) в мобилното приложение за разделяне на UI от бизнес логиката, CQRS (Command Query Responsibility Segregation) в Services слоя за разделяне на операциите за запис от операциите за четене, Repository Pattern в DataLayer за абстракция на достъпа до базата данни, MediatR за изпращане на команди и заявки между контролерите и Handler-ите, както и Dependency Injection навсякъде за управление на зависимостите между компонентите.

3.2.1. Клиентски слой (Presentation Layer)

Клиентският слой е реализиран чрез .NET MAUI приложение за Android. Той отговаря за визуализацията на данните и взаимодействието с потребителя. Приложението използва MVVM (Model-View-ViewModel) архитектурен шаблон, който осигурява ясно разделяне между потребителския интерфейс и бизнес логиката.

Комуникацията със сървъра се осъществява чрез RESTful API с HTTP заявки, използвайки HttpClient класа от .NET. Данните се обменят в JSON формат. Класът ApiService централизира всички API извиквания и предоставя типизирани методи за GET, POST операции.

3.2.2. Сървърен слой (Business Logic Layer)

Сървърният слой е реализиран като ASP.NET Core Web API проект. Той отговаря за обработката на бизнес логиката, валидацията на данните и комуникацията с базата данни. Вътрешно слоят е разделен на три проекта:

CalorieCalculator.API – съдържа контролерите, които приемат HTTP заявки и ги насочват към съответните обработчици. Контролерите не съдържат бизнес логика, те делегират обработката чрез MediatR.

Services – съдържа командите (Commands), заявките (Queries) и техните обработчици (Handlers), реализиращи CQRS шаблона. Тук се намира и AuthService за автентикация, както и ServiceManager, той обединява три отделни сървиса в една точка за достъп, така че контролерите и обработчиците да не инжектират всяка услуга поотделно.

DataLayer – съдържа моделите на данните (Entities), ApplicationDbContext за Entity Framework Core, репозитори интерфейса (IRepository) и системата за миграции (MigrationRunner). SQL миграциите се съхраняват като embedded resources.

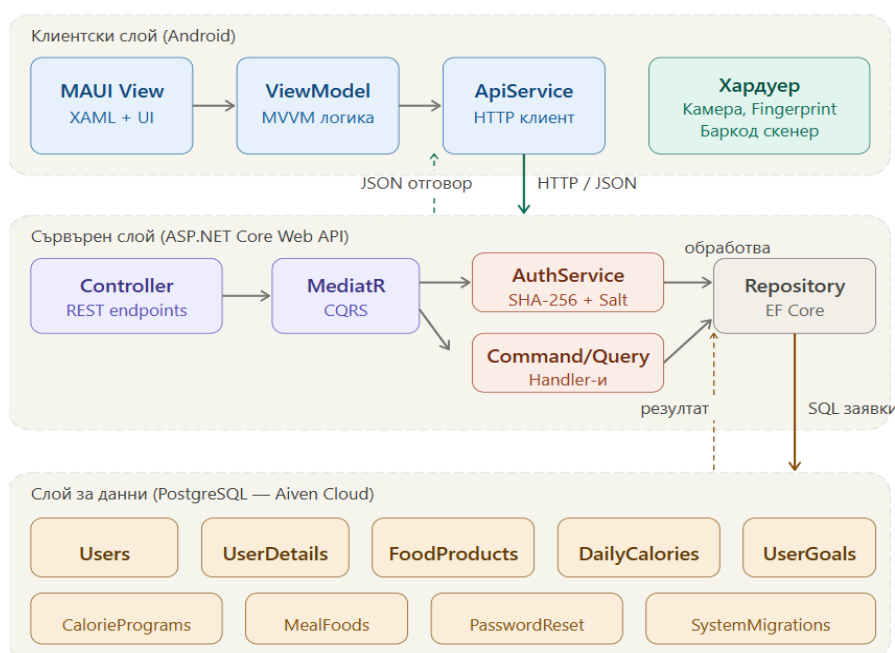
3.2.3. Слой за данни (Data Layer)

Базата данни е PostgreSQL, хоствана в облака чрез услугата Aiven. Връзката е защитена с SSL (SSL Mode = Require). Достъпът до базата данни се осъществява чрез Entity Framework Core с Npgsql доставчик.

За управление на схемата на базата данни е разработена собствена система за миграции (MigrationRunner), която при стартиране на API проекта автоматично проверява и изпълнява нови SQL миграции.

3.3. Архитектурна диаграма

Комуникационният поток между слоевете има следният вид:



Фиг. 1 - Диаграма за архитектурата на CalorieCalculator

Клиентски слой – MAUI View изпраща заявки към ViewModel, който чрез ApiService комуникира с API-то по HTTP/JSON. Хардуерните функции (камера, fingerprint, баркод) са отделен модул. Изпраща HTTP заявка към API сървъра, а резултатите от сървърният слой му се връщат в JSON формат.

Сървърен слой – Controller приема заявката и я делегира чрез MediatR към два типа обработчици: AuthService (за автентикация с SHA-256 хеширане) и Command/Query Handler-и (за бизнес операции). И двата достъпват базата чрез Repository (EF Core).

Слой за данни – PostgreSQL в Aiven Cloud с всички таблици. Комуникират с Repository чрез SQL заявки, обработват ги и връщат резултат. След това приложението десериализира отговора и актуализира потребителския интерфейс.

3.4. Модел на данните

Базата данни съдържа следните основни таблици:

Таблица	Описание
Users	Потребители – имейл, хеш на парола, дата на регистрация, статус на акаунта
UserDetails	Детайли за профила – псевдоним, пол, възраст, височина, тегло, ниво на активност, цел, желано тегло
UserGoals	Цели на потребителя – вид цел и желано тегло
FoodCategories	Категории храни – месо, риба, млечни, зърнени, растителни и др.
FoodProducts	Хранителни продукти – име, калории, протеин, въглехидрати, мазнини, грамаж, баркод
CaloriePrograms	Дневни калорийни планове – калории, макроси, дата, потребител
CalorieProgramMeals	Ястия в калориен план – закуска, обяд, вечеря, снакс
MealFoods	Храни в ястие – продукт, грамаж, връзка към ястие
SystemMigrations	Изпълнени миграции – име на файл, дата на изпълнение

PasswordResetCodes	Кодове за възстановяване на пароли – уникален 6-цифрен код при забравена парола
---------------------------	---

Таблица 2 - Таблиците в проекта и предназначението им

3.4.1. Връзки между таблиците

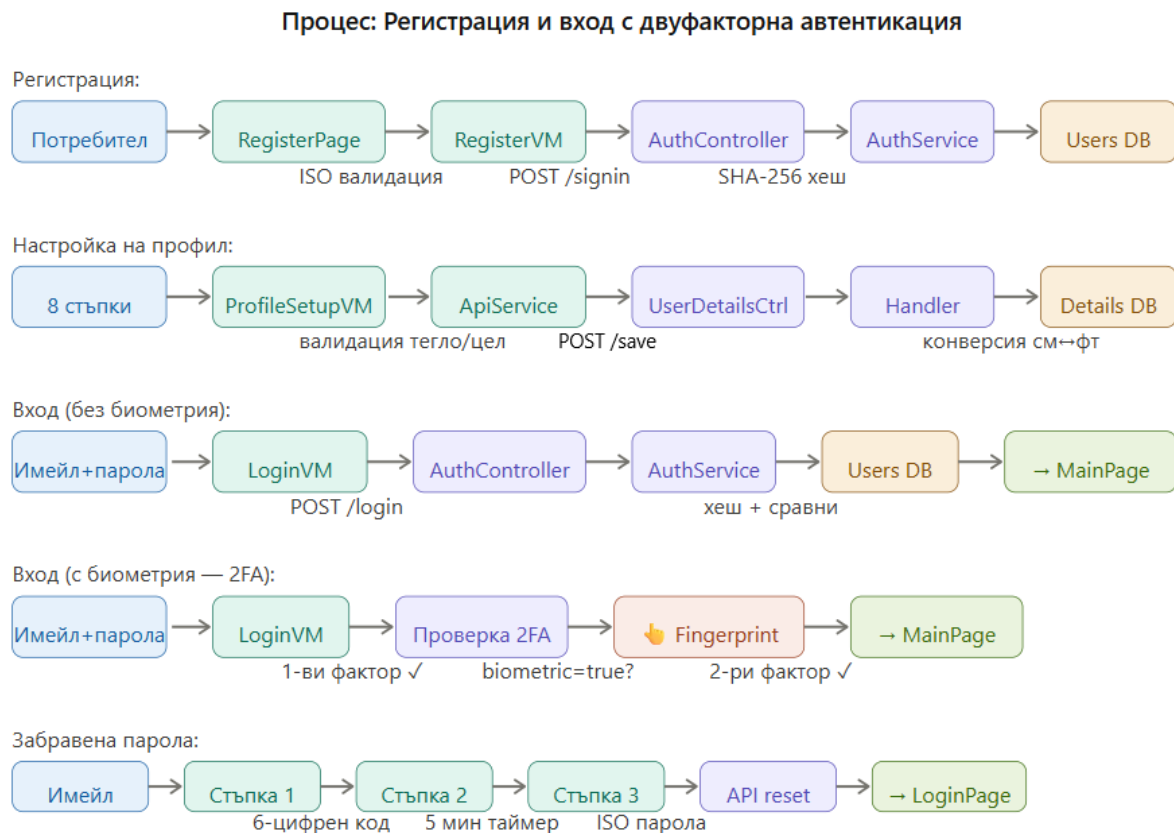
Моделът на данните използва следните връзки:

- Users → UserDetails: един към един – всеки потребител има един профил с детайли.
- Users → UserGoals: един към много – всеки потребител може да си поставя различни цели, като всяка цел принадлежи на един потребител.
- Users → CaloriePrograms: един към много – всеки потребител има множество дневни планове.
- Users → PasswordResetCodes: един към много – за един потребител могат да бъдат генерирани няколко кода за възстановяване на парола.
- CaloriePrograms → CalorieProgramMeals: един към много – всеки план съдържа до четири ястия.
- CalorieProgramMeals → MealFoods: един към много – всяко ястие съдържа множество храни.
- FoodCategories → FoodProducts: един към много – всяка категория съдържа множество продукти.
- FoodProducts → MealFoods: един към много – един продукт може да участва в много ястия.



Фиг. 2 - ERD (Entity Relationship Diagram) на таблиците в базата данни

3.5. Процеси в системата

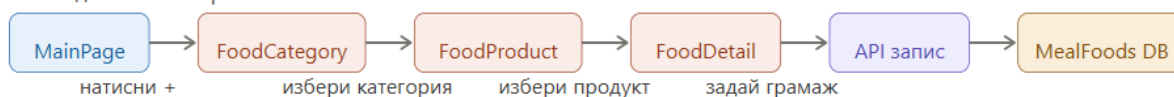


Фиг. 3 - Схема на процесите при регистрация/вход с двуфакторна автентикация

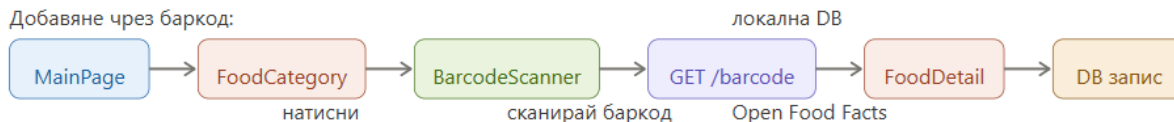
Схема фиг.3 – Регистрация и вход с двуфакторна автентикация: Показва пет различни потока на регистрацията – нормална регистрация (с ISO валидация → SHA-256 хеширане → Users DB), настройка на профил (8 стъпки → валидация тегло/цел → конверсия мерни единици → UserDetails DB), вход без биометрия (имейл+парола → хеш+сравнение → MainPage) и вход с 2FA (първи фактор парола → проверка biometric=true → fingerprint сензор → MainPage), плюс забравена парола (3 стъпки с таймер – 6-цифрен код, 5 минутен таймер и нова парола по ISO).

Процес: Добавяне на храна и проследяване на калории

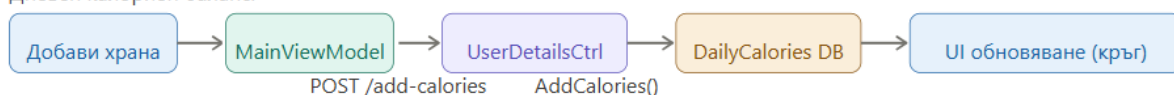
Ръчно добавяне на храна:



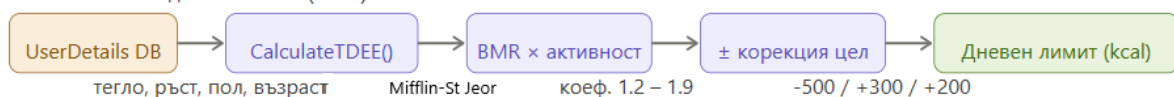
Добавяне чрез баркод:



Дневен калориен баланс:



Изчисляване на дневен лимит (TDEE):

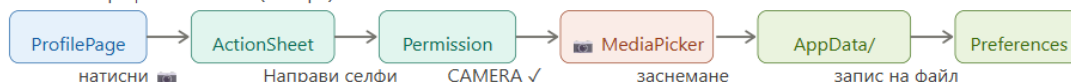


Фиг. 4 - Схема на процесите при добавяне на храна и проследяване на калориите

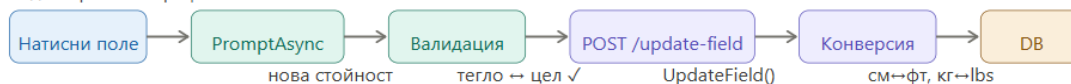
Схема фиг. 4 – Добавяне на храна и проследяване на калории: Показва ръчно добавяне (MainPage → категории → продукти → детайл с грамаж → DB), добавяне чрез баркод (скенер → локална DB или Open Food Facts API → детайл → DB), дневен калориен баланс (добавяне → AddCalories() → DailyCalories DB → UI кръг) и TDEE изчисление (UserDetails → Mifflin-St Jeor → × активност → ± цел → = дневен лимит).

Процес: Профил, календар и хардуерни функции

Смяна на профилна снимка (камера):



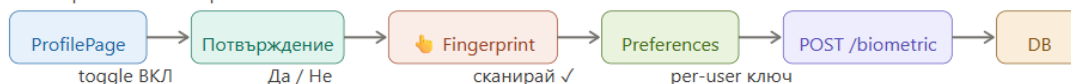
Редактиране на профилно поле:



Калориен календар:



Активиране на биометрия:



Изход от акаунта:



Фиг. 5 - Схема на процесите при профил/календар страница със съответните хардуерни функции

Схема фиг.5 – Профил, календар и хардуерни функции: Показва смяна на снимка (📷 → Permission → MediaPicker → AppData/ → Preferences), редактиране на поле (prompt → валидация → POST /update-field → конверсия → DB), календар (GET /monthly → TDEE + дневни данни → ArcConverter → зелен/червен пръстен), активиране на биометрия (toggle → потвърждение → fingerprint → Preferences per-user → POST /biometric → DB) и logout (потвърждение → изчисти session → запази bio+photo+email → LoginPage).

3.6. Проектиране на потребителския интерфейс

Потребителският интерфейс е проектиран с фокус върху простотата и удобството на използване. Навигацията е организирана чрез долна навигационна лента (Bottom Navigation Bar) с четири раздела:

Раздел	Страница	Функционалност
Profile	ProfilePage	Потребителски профил и настройки
Today	MainPage	Дневен калориен план с ястия и макроси
Progress	ProgressPage	Проследяване на напредъка
Calendar	CalendarPage	Месечен преглед на калорийните данни

Таблица 3 - Четирите основни страници в приложението и функционалностите им

Навигационната лента е реализирана като отделен компонент (BottomNavBar), който се използва повторно във всички основни страници. При натискане на раздел иконата и текстът се оцветяват в тюркоазен цвят (#4DB6AC), а неактивните остават сиви (#BDBDBD).

3.6.1. Навигационна карта

Пълната навигационна структура на приложението е:

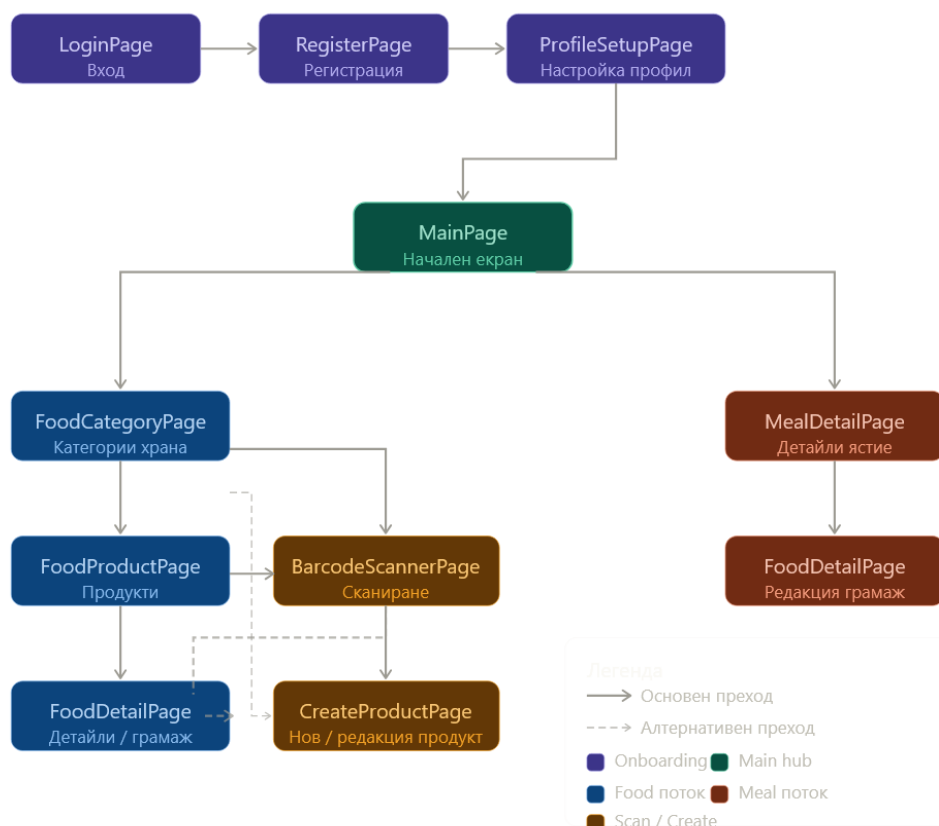
LoginPage → RegisterPage → ProfileSetupPage → MainPage

MainPage → FoodCategoryPage → FoodProductPage → FoodDetailPage

MainPage → MealDetailPage → FoodDetailPage (редактиране на грамаж)

FoodCategoryPage → BarcodeScannerPage → FoodDetailPage / CreateProductPage

FoodCategoryPage / FoodProductPage → CreateProductPage (нов/редакция)



Фиг. 6 - Навигационна структура на приложението

3.7. Формули за изчисляване на калории

Приложението използва уравнението на Mifflin-St Jeor за изчисляване на базалния метаболизъм (BMR), което е научно доказано като най-точното за тази цел.

3.7.1. Базален метаболизъм (BMR)

За мъже: $BMR = (10 \times \text{тегло в кг}) + (6.25 \times \text{височина в см}) - (5 \times \text{възраст}) + 5$

За жени: $BMR = (10 \times \text{тегло в кг}) + (6.25 \times \text{височина в см}) - (5 \times \text{възраст}) - 161$

3.7.2. Дневен енергиен разход (TDEE)

TDEE се изчислява като BMR, умножен по коефициент за ниво на активност:

Ниво на активност	Коефициент
Заседнал начин на живот	1.2
Леко активен (1-3 пъти/седмица)	1.375
Умерено активен (3-5 пъти/седмица)	1.55
Много активен (6-7 пъти/седмица)	1.725
Изключително активен	1.9

Таблица 4 - Ниво на активността и коефициента, по който се умножава във формулата за TDEE

3.7.3. Корекция спрямо целта

Отслабване: TDEE – 500 kcal (загуба на ~0.5 кг/седмица)

Поддържане: TDEE (без промяна)

Наддаване на тегло: TDEE + 300 kcal

Набиране на мускулна маса: TDEE + 400 kcal

3.7.4. Разпределение на макронутриенти

Протеинът се изчислява като грамове на килограм телесно тегло (1.6–2.2 г/кг в зависимост от целта). Мазнините заемат 25–30% от общите калории. Остатъкът се разпределя към въглехидрати.

4. Реализация

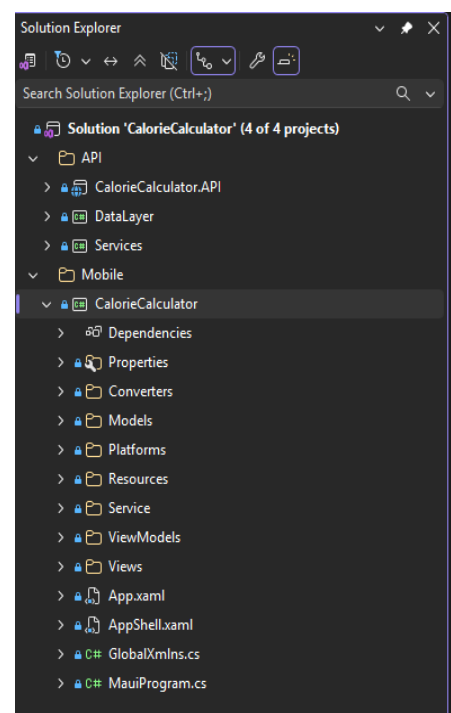
4.1. Структура на проекта

Решението е организирано в четири отделни проекта в рамките на едно Visual Studio решение (VS Solution):

4.1.1. CalorieCalculator (MAUI)

Мобилното приложение е структурирано по следния начин:

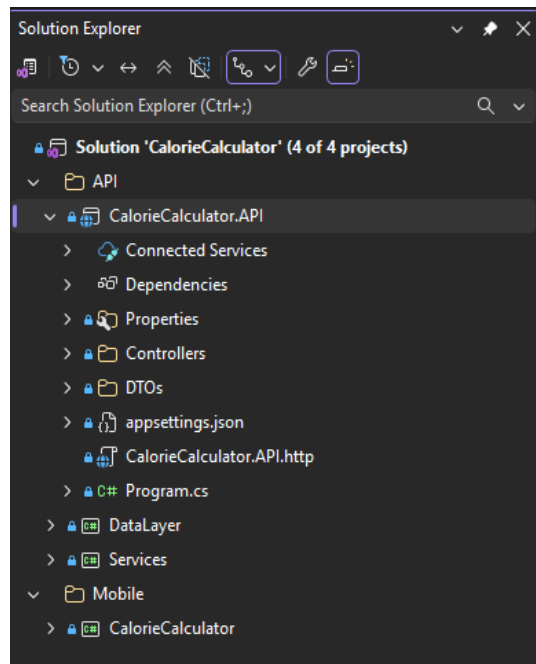
- Views/ – XAML файлове за потребителския интерфейс
- ViewModels/ – класове, реализиращи `INotifyPropertyChanged` за свързване с изгледите
- Models/ – DTO класове за десериализация на API отговори
- Services/ – `ApiService` за HTTP комуникация
- Controls/ – преползваеми компоненти (например `BottomNavBar`)
- Helpers/ – помощни класове (например `UserSession`)



Фиг. 7 – Структура на CalorieCalculator(MAUI)

4.1.2. CalorieCalculator.API

API проектът съдържа контролерите, които приемат HTTP заявки. Всеки контролер инжектира `IServices` и използва `Mediator` за изпращане на команди и заявки към обработчиците. При стартиране на приложението, `Program.cs` конфигурира `dependency injection` контейнера, регистрира `DbContext`, `MediatR`, `Repository` и `AuthService`, след което стартира `MigrationRunner` за автоматично изпълнение на нови SQL миграции.

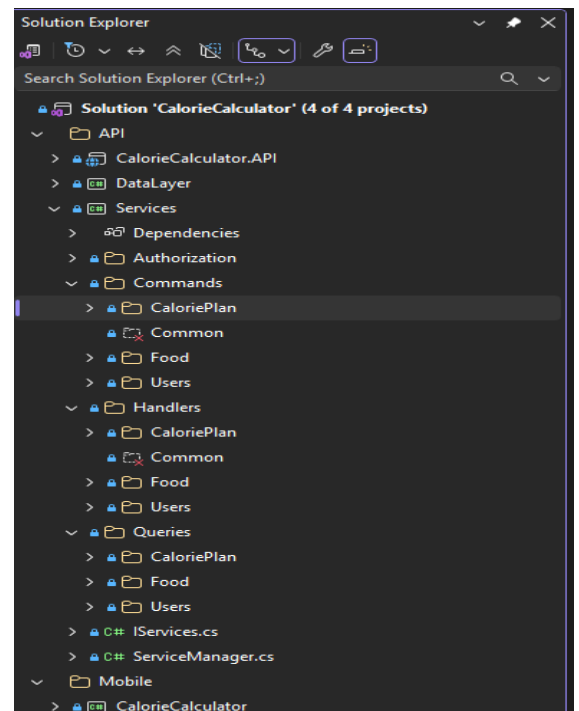


Фиг. 8 – Структура на CalorieCalculator.API

4.1.3. Services

Проектът за услуги съдържа бизнес логиката, организирана по CQRS(Command Query Responsibility Segregation) шаблона:

- Commands/ – класове, описващи операции за промяна на данни (например FoodProductCommand, AddFoodToMealCommand).
- Queries/ – класове, описващи операции за четене на данни (например GetDailyProgramQuery, GetAllCategoriesQuery).
- Handlers/ – обработчици, които изпълняват бизнес логиката за всяка команда или заявка.



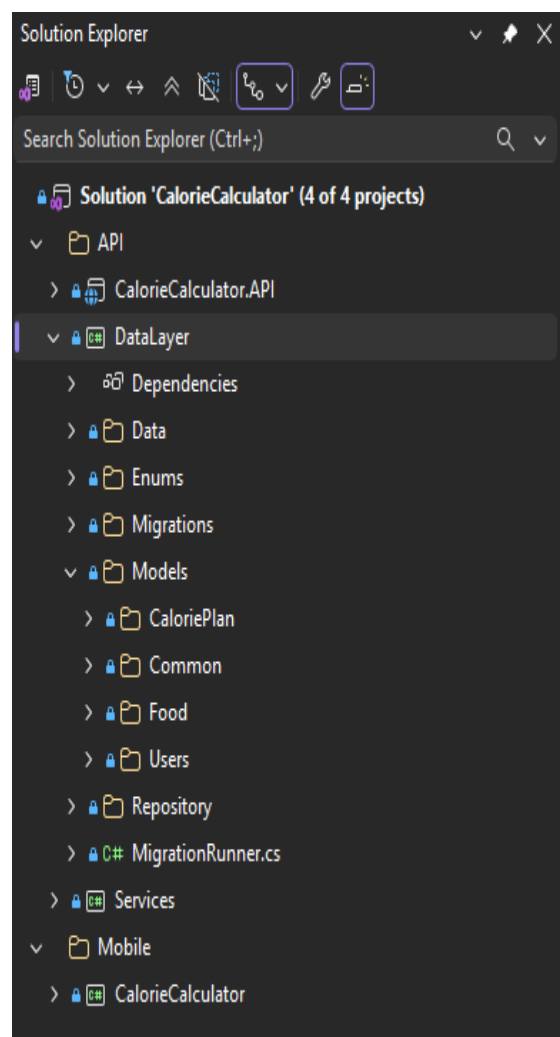
Фиг. 9 – Организация на Services

Интерфейсът `IServices` обединява `IRepository`, `IMediator` и `IAuthService` в единна точка за достъп, реализирана от класа `ServiceManager`.

4.1.4. DataLayer

Слоят за данни съдържа:

- `Models/` – Entity класове с private setters и фабрични методи (`Create`) и методи за актуализация (`Update`). Този подход осигурява капсулация и контролиран достъп до свойствата.
- `Data/AppDbContext.cs` – конфигурация на Entity Framework Core контекста с `DbSet` свойства за всяка таблица.
- `Repositories/` – генеричен `IRepository` интерфейс с методи `Set<T>()`, `SetNoTracking<T>()`, `Add()`, `Update()`, `Delete()`, `Save()`, `BeginTransaction()` и `ExecuteSqlRawAsync()`.
- `Migrations/` – SQL файлове с номерирана номенклатура по дати и по конкретно свойство на изпълнение (`20260405_CREATE...`, `20260405_ALTER...`, `20260405_UPDATE...`), вградени като `Embedded Resources`.



Фиг. 10 – Организация на `DataLayer`

4.2. Реализация на автентикацията

Системата за автентикация използва SHA-256 хеширане с уникална сол (`Salt`) за всеки потребител. Солта представлява произволни (случайни) данни, които се добавят към входната информация (например парола) преди тя да бъде хеширана. Самият процес на регистрация преминава през следните стъпки:

1. Потребителят въвежда имейл и парола.
2. Генерира се случайна 32-байтова сол (Salt) чрез RandomNumberGenerator.
3. Паролата се конкатенира със солта и се хешира чрез SHA-256.
4. Хеширането се повтаря 10 000 пъти за допълнителна защита срещу brute-force атаки.
5. Хешът и солта се съхраняват в базата данни. Оригиналната парола никога не се записва там.

При вход в системата, въведената парола се хешира по същия начин с извлечената от базата данни сол и полученият хеш се сравнява със съхранения. При съвпадение входът е успешен.

4.2.1. ISO/IEC 27001 валидация на пароли (Красимир)

Освен стандартното хеширане, паролите преминават през реално-времева валидация спрямо изискванията на международния стандарт ISO/IEC 27001 за информационна сигурност. При въвеждане на паролата потребителят вижда динамични индикатори (зелена отметка, когато е изпълнил даденото условие по ISO/IEC 27001 стандарта, или сива отметка, когато не е изпълнил даденото условие от стандарта). Това важи за всяко от следните изисквания: минимум 8 символа, поне една главна буква (A-Z), поне една малка буква (a-z), поне една цифра (0-9) и поне един специален символ (!@#\$...). Бутонът за регистрация се активира едва когато всички изисквания са изпълнени. Имейл адресът се валидира чрез регулярен израз и се изисква да съдържа само малки букви – при въвеждане на главна буква системата показва грешка вместо автоматично преобразуване, за да е наясно потребителят с точния имейл адрес, който използва.

4.2.2. Забравена парола – тристъпков wizard (Красимир)

Процесът за възстановяване на парола е реализиран като wizard от три стъпки. На първата стъпка потребителят въвежда имейла, с който се е регистрирал, и API генерира 6-цифрен код, който се показва чрез DisplayAlert (симулация на имейл изпращане). Кодът се съхранява в таблицата PasswordResetCodes с еднократна употреба и изтичане след 5 минути. На втората стъпка потребителят въвежда 6-цифрения код в шест отделни полета с автоматичен фокус (auto-focus) между тях и визуален обратен брояч от 5 минути. При изтичане на времето се предоставя опция за повторно изпращане на код.

На третата стъпка потребителят задава нова парола, която преминава през същата ISO/IEC 27001 валидация, и потвърждава паролата.

4.2.3. Двухфакторна биометрична автентикация с пръстов отпечатък (Красимир)

Нашето приложение CalorieCalculator предлага двухфакторна автентикация (2FA) с пръстов отпечатък, реализирана чрез NuGet пакета Plugin.Fingerprint (v2.1.5). Двухфакторната автентикация се активира по желание от потребителя, ако той иска да защити личните си данни по по-сигурен начин. Тази функционалност използва хардуерния биометричен сензор на Android устройството, което я прави една от ключовите хардуерни функции, използвани в проекта.

Включване на биометрията: Потребителят може да активира биометричния вход от профилната страница чрез toggle бутон. При натискане системата проверява дали устройството поддържа биометрия, пита за потвърждение и изисква успешно сканиране на пръстов отпечатък. Настройката се записва в Preferences с уникален ключ per-user (biometric_enabled_{userId}) и в базата данни (колона IsBiometricEnabled в таблицата Users). При нова регистрация биометрията е винаги изключена по подразбиране.

Вход с биометрия: При вход в системата, след успешна проверка на имейл и парола (първи фактор), системата проверява дали потребителят е активирал биометрия. Ако да – Android показва системния fingerprint dialog (втори фактор). При успешно сканиране потребителят влиза в приложението. При неуспех се показва съобщение за грешка и възможност за повторен опит. Различните потребители на едно устройство могат да имат различни настройки за биометрия.

Техническа реализация: В MainActivity.OnCreate() се извиква CrossFingerprint.SetCurrentActivityResolver(() => this) за инициализация на Plugin.Fingerprint. Статичният клас BiometricAuthenticator предоставя методите AuthenticateAsync() и IsAvailableAsync(). За Android са необходими permissions USE_BIOMETRIC и USE_FINGERPRINT в AndroidManifest.xml. Също така за Android е необходимо и устройството ти да поддържа биометрия и да има сензор за пръстови отпечатъци. На Windows при пожелание може да се симулира чрез DisplayAlert диалог за тестване.

4.3. Реализация на лични настройки и статистика

4.3.1. Профилна снимка чрез селфи камерата (Маргарита)

Потребителят може да смени профилната си снимка чрез натискане на камера иконката върху аватара в профилната страница. Тази функционалност използва хардуерната предна камера на устройството, което я прави втората ключова хардуерна функция на проекта (след биометричния сензор). Използва се MAUI MediaPlayer API (MediaPlayer.Default.CapturePhotoAsync()) за заснемане на снимка в реално време. Достъпна е само опцията за правене на селфи – без избор от галерия, за да се гарантира автентичност на профилната снимка.

Снимката се запазва в локалната директория на приложението (FileSystem.AppDataDirectory) с уникално име, включващо потребителския идентификатор и timestamp (profile_{userId}_{timestamp}.jpg), за да се избегне кеширане от MAUI. Пътят до снимката се съхранява в Preferences с ключ profile_photo_path_{userId}, което осигурява, че различните потребители на едно устройство имат отделни профилни снимки. При смяна на снимка старият файл се изтрива автоматично. Снимката се запазва след logout и се зарежда при следващо влизане. По подразбиране аватарът е цветен кръг (син за мъже, розов за жени) с първата буква от псевдонима, който си е избрал потребителя.

За Android се изисква CAMERA permission, което се заявява програматично преди отваряне на камерата чрез Permissions.RequestAsync<Permissions.Camera>(). При първо използване на камерата на устройството, Android показва системен диалог за разрешение, след което камерата се отваря нормално.

4.3.2. Редактиране на профилни детайли (Маргарита)

В профилната страница полетата Ръст, Тегло, Активност, Цел и Желано тегло са редактируеми чрез натискане. При натискане на поле изскача prompt диалог (DisplayPromptAsync) за числови стойности или ActionSheet за избор от списък (Активност, Цел). Промяната се записва в базата данни чрез POST api/UserDetails/update-field, като API методът UpdateField в модела UserDetails автоматично конвертира между мерните единици – например при промяна на HeightCm се преизчислява HeightFt и обратно. Полетата Пол и Възраст не са редактируеми. При редактиране на тегло или

желано тегло се извършва валидация спрямо текущата цел – системата не позволява несъответствия (например желано тегло по-голямо от текущото при цел „Загуба на тегло“). Мерните единици се показват спрямо избора на потребителя при регистрация.

4.3.3. Калорийни пръстени в календара (Маргарита)

Около всеки ден в календара със записани калории се рисува калориен пръстен – зелена дъга, която започва от 12 часа и расте по часовниковата стрелка, пропорционално на процента изядени калории спрямо дневния лимит на потребителя. При 50% прогрес се вижда само дясната половина на пръстена. Ако калориите надвишат лимита (над 100%), пръстенът се запълва изцяло и става червен. Конверсията на прогреса в дъга се извършва чрез custom конвертор `ProgressToArcPathConverter`, който генерира `PathGeometry` с `ArcSegment` за съответния ъгъл. Под пръстена е сив фонов кръг, показващ 100% от лимита. Днешният ден в календара е маркиран със запълнен син кръг, а избраният ден – само с тънка синя рамка без запълване. При натискане на ден под календара се показва информация: колко са изядените калории от потребителя на този ден, дневния му лимит и съответно дали е спазил поставената си цел.

4.4. Реализация на калорийни планове (Мартин)

Калорийният план е централният елемент на приложението – той определя дневните хранителни цели на потребителя и служи като контейнер за ястията и храните.

4.4.1. Създаване и автоматично попълване

Калорийните планове се създават чрез `HandlerCalorieProgram` при всеки вход в системата. Обработчикът проверява кога е създаден последният план и ако има пропуснати дни, автоматично генерира планове за всеки от тях до днешна дата. Изчислението използва актуалните данни от `UserDetails` и уравнението на Mifflin-St Jeor. Резултатът се записва в таблицата `CaloriePrograms` с полета `CaloriesPerDay`, `ProteinPerDay`, `CarbsPerDay`, `FatsPerDay`, `ProgramDate` и `UserID`.

4.4.2. Структура на ястията

Всеки план съдържа четири типа ястия (закуска, обяд, вечеря, снакс), но те не се създават заедно с плана. API винаги връща четири ястия с флаг `Exists`, а реалният запис в

CalorieProgramMeals се създава едва при добавяне на първата храна. Това избягва празни записи в базата данни.

4.4.3. Навигация между планове

Стрелки на главния екран позволяват преглед на предишни и следващи дневни планове. Обработчикът връща флагове HasPrevious и HasNext, определящи видимостта на стрелките. Потребителят може да се върне назад и да добави пропуснати храни.

4.5. CRUD операции върху храни в ястие (MealFoods) (Мартин)

4.5.1. Добавяне

Потребителят избира продукт чрез категория или търсене, въвежда грамаж на FoodDetailPage и натиска „Добави към ястие“. Обработчикът FoodToMealCommand създава ястието ако не съществува, след което добавя запис в MealFoods с ProductID и Weight в транзакция.

4.5.2. Преглед

MealDetailPage показва всички храни в ястие с калории и макроси, изчислени спрямо записания грамаж на сървъра.

4.5.3. Редактиране на грамаж

Иконата „молив“ отваря FoodDetailPage с попълнен текущ грамаж. Същият обработчик FoodToMealCommand проверява дали е подаден MealFoodID – ако да, обновява грамажа; ако не, добавя нова храна. Един ендпойнт обслужва и двете операции.

4.5.4. Изтриване

Бутонът „X“ премахва храната незабавно без потвърждение чрез DELETE заявка. Списъкът и обобщените стойности се обновяват автоматично.

4.6. CRUD операции върху хранителни продукти (FoodProducts) (Мартин)

4.6.1. Създаване и редактиране

Един обработчик `FoodProductCommand` обслужва и двете операции. При липса на `ProductID` създава нов продукт; при наличие – извлича съществуващия и извиква `Update()`. Същият формуляр `CreateProductPage` се използва и за двата случая, като заглавието и бутонът се променят динамично. При създаване чрез баркод скенер полето `Barcode` също се записва.

4.6.2. Преглед

Продуктите се визуализират по категории с търсене в реално време. API поддържа филтриране по `CategoryID` и незадължителен `search` параметър.

4.7. Баркод скенер (Мартин)

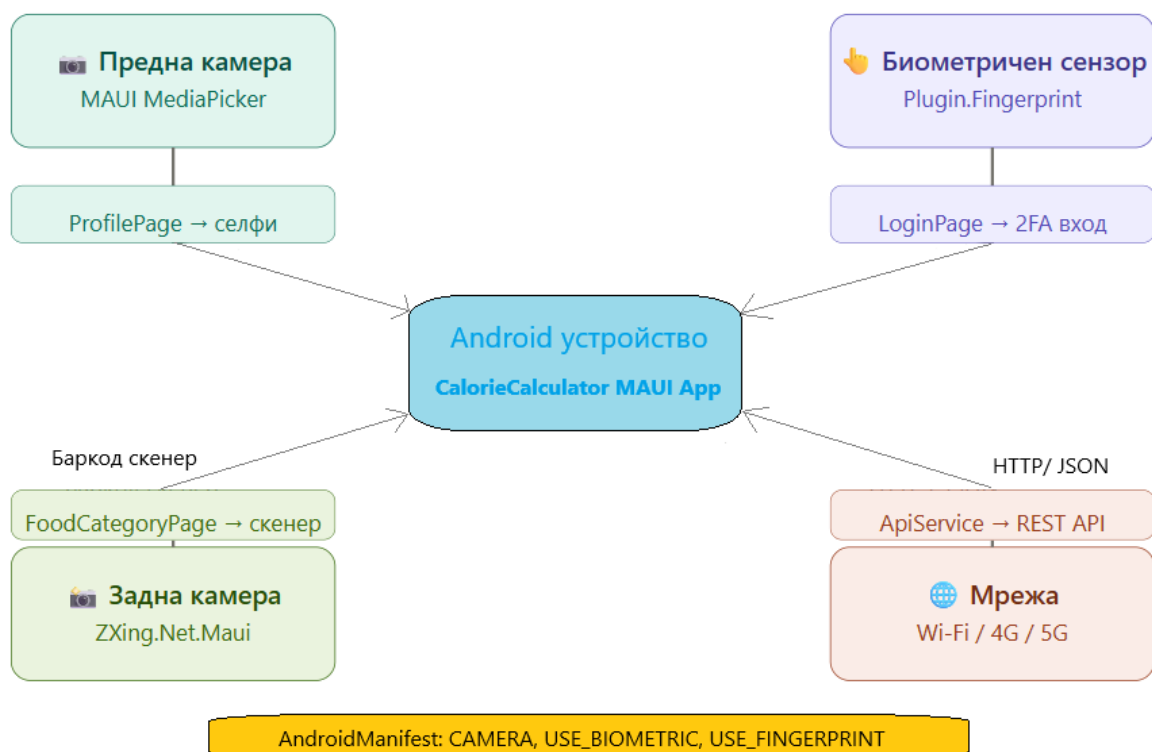
Функцията за сканиране на баркод е реализирана чрез библиотеката `ZXing.Net.Maui`, която използва камерата на устройството за разпознаване на EAN-13, EAN-8 и UPC баркодове. Потокът на работа е следният:

1. Потребителят натиска иконата за сканиране в страницата с категориите на различните по вид храни.
2. Камерата се активира и сканира баркода.
3. API проверява дали баркодът съществува в локалната база данни (колона `Barcode` в таблица `FoodProducts`).
4. Ако не е намерен локално, се изпраща заявка към `Open Food Facts API`.
5. При намиране онлайн, формулярът за създаване на продукт се попълва автоматично с данните от API.
6. След запис баркодът се съхранява в колоната `Barcode` на продукта за бъдещи сканирания.

Чрез тази трета функционалност потребителите, лесно и удобно, само чрез едно кликване на камерата могат да въведат какво са яли днес, стига съответният продукт да притежава баркод.

4.8. Мрежова свързаност (Wi-Fi / мобилни данни)

Цялата комуникация между мобилното приложение и API сървъра се осъществява чрез HTTP/HTTPS протокол през Wi-Fi или мобилна мрежа. ApiService използва HttpClient за изпращане на заявки към REST API-то и получаване на JSON отговори. Данните се сериализират и десериализират чрез System.Text.Json. При липса на мрежова връзка приложението показва съобщение „Няма връзка със сървъра. Провери интернет връзката.“



Фиг. 11 - Схема, описваща връзките с хардуерните устройства

4.9. Модели на данните (Entities)

Всички модели в `DataLayer` следват принципа на капсулация – свойствата имат `private` setters и се променят единствено чрез публични мутатор методи. Това гарантира, че бизнес правилата се спазват при всяка промяна на данните.

User (таблица Users)

Моделът `User` представлява потребителски акаунт в системата. Полета: `Id` (`Guid`, първичен ключ, генерира се автоматично), `Email` (`string`, уникален имейл адрес на потребителя), `PasswordHash` (`string`, хеширана парола с SHA-256), `Salt` (`string`, уникална сол за хеширане, генерирана при регистрация), `IsBiometricEnabled` (`bool`, указва дали потребителят е активирал двуфакторна биометрична автентикация), `IsActive` (`bool`, статус на акаунта), `LastPasswordLogin` (`DateTime`, дата и час на последното влизане с парола), `ActivationDate` (`DateTime`, дата на активиране на акаунта), `CreatedAt` (`DateTime`, дата на създаване) и `UpdatedAt` (`DateTime?`, дата на последна промяна).

Мутатор методи: *`Update(string email, string passwordHash, string salt)`* – задава основните данни при регистрация, включително имейл, хеширана парола и сол. *`SetBiometric(bool enabled)`* – включва или изключва биометричната автентикация и обновява `UpdatedAt`. *`UpdatePassword(string passwordHash, string salt)`* – задава нова хеширана парола и нова сол при смяна на парола чрез wizard-а за забравена парола, обновява `UpdatedAt`. *`UpdateLastPasswordLogin()`* – обновява датата и часа на последното успешно влизане с парола до текущия момент в UTC и автоматично актуализира полето `UpdatedAt`. *`Deactivate()`* – променя статуса на потребителя на неактивен, като задава стойност `false` на свойството `IsActive` и обновява `UpdatedAt`. Използва се при деактивиране на акаунт.

UserDetails (таблица UserDetails)

Моделът `UserDetails` съхранява телесните данни и целите на потребителя. Полета: `Id` (`Guid`, първичен ключ), `UserId` (`Guid`, външен ключ към `Users`), `Nickname` (`string`, псевдоним на потребителя, минимум 2 символа), `Gender` (`enum Gender` – 1=Мъж, 2=Жена), `DateOfBirth` (`DateTime`, дата на раждане, използва се за изчисляване на възрастта), `HeightCm` (`decimal?`, ръст в сантиметри), `HeightFt` (`decimal?`, ръст във футове, изчислява се автоматично от `HeightCm` и обратно), `WeightKg` (`decimal?`, тегло в килограми), `WeightLbs` (`decimal?`, тегло в паундове, изчислява се автоматично от

WeightKg и обратно), ActivityLevel (enum ActivityLevel – 1=Заседнал, 2=Леко активен, 3=Умерено активен, 4=Много активен, 5=Изключително активен), CurrentGoal (enum GoalType – 1=Загуба на тегло, 2=Задържане, 3=Качване на тегло, 4=Качване на мускулна маса), TargetWeightKg (decimal?, желано тегло в килограми), TargetWeightLbs (decimal?, желано тегло в паундове, конвертира се автоматично), CreatedAt (DateTime) и UpdatedAt (DateTime?).

Мутатор методи: *Update(string nickname, int gender, DateTime dateOfBirth, decimal? heightCm, decimal? heightFt, decimal? weightKg, decimal? weightLbs, int activityLevel, int currentGoal, decimal? targetWeightKg, decimal? targetWeightLbs)* – основен метод за запис на всички профилни данни. При подаване на HeightCm автоматично изчислява HeightFt по формулата HeightCm / 30.48 и обратно. Същата автоматична конверсия се прилага за WeightKg/WeightLbs (коефициент 2.20462) и TargetWeightKg/TargetWeightLbs. *UpdateNickname(string newNickname)* – променя псевдонима и обновява UpdatedAt. *UpdateField(string fieldName, string value)* – универсален метод за промяна на отделно поле чрез име на полето (string). *CalculateTDEE()* – изчислява максималния дневен прием на калории спрямо телесните данни по формулата на Mifflin-St Jeor:

- за мъже: $BMR = 10 \times \text{тегло(кг)} + 6.25 \times \text{ръст(см)} - 5 \times \text{възраст} - 161$
- за жени: $BMR = 10 \times \text{тегло(кг)} + 6.25 \times \text{ръст(см)} - 5 \times \text{възраст} + 5$.

BMR се умножава по коефициент на активност (1.2, 1.375, 1.55, 1.725 или 1.9) и се коригира спрямо целта (-500 за отслабване, +300 за качване, +200 за мускулна маса, без промяна за поддържане).

UserGoal (таблица UserGoals)

Моделът UserGoal представлява конкретна цел на потребителя с времеви период. Полета: Id (Guid, първичен ключ), UserId (Guid, външен ключ към Users), GoalType (enum GoalType – същите стойности като CurrentGoal), TargetWeightKg (decimal?), TargetWeightLbs (decimal?), StartDate (DateTime, начало на периода за целта), EndDate (DateTime, край на периода), IsActive (bool, дали целта е текущо активна) и CreatedAt (DateTime).

Мутатор методи: *Update (int goalType, decimal? targetWeightKg, decimal? targetWeightLbs, DateTime startDate, DateTime endDate)* – задава типа на целта, целевото

тегло и времеви период. *GetCalorieTarget(decimal?)* – метод, който определя дневния калориен прием, като коригира предоставения TDEE спрямо вида на избраната цел.

PasswordResetCode (таблица PasswordResetCodes)

Моделът PasswordResetCode съхранява кодове за възстановяване на парола. Полета: Id (Guid, първичен ключ), UserId (Guid, външен ключ към Users), Code (string, 6-цифрен код), ExpiresAt (DateTime, дата на изтичане – 5 минути след генериране), IsUsed (bool, дали кодът вече е използван) и CreatedAt (DateTime).

Мутатор методи: *MarkAsUsed()* маркира кода като използван, за да не може да бъде употребен повторно. *IsValid()* – проверява дали кодът не е изтекъл (*ExpiresAt > DateTime.UtcNow*) и не е използван (*IsUsed == false*).

DailyCalories (таблица DailyCalories)

Моделът DailyCalories съхранява дневния калориен прием на потребителя. Полета: Id (Guid, първичен ключ), UserId (Guid, външен ключ към Users), Date (DateTime, датата за която е записът, с уникален индекс UserId+Date, за да се гарантира един запис на потребител на ден), CaloriesEaten (int, общо изядени калории за деня), CreatedAt (DateTime) и UpdatedAt (DateTime?).

Мутатор методи: *AddCalories (int calories)* – добавя калории към текущия дневен запис (*CaloriesEaten += calories*) и обновява UpdatedAt. Използва се при добавяне на храна от MainPage. *SetCalories (int calories)* – задава точна стойност на калориите (замества текущата) и обновява UpdatedAt. Използва се при ръчна корекция.

FoodProduct (таблица FoodProducts)

Моделът FoodProduct представлява хранителен продукт в базата данни. Полета: Id (Guid, първичен ключ), CategoryId (Guid, външен ключ към FoodCategories), Name (string, име на продукта), Description (string?, незадължително описание), Calories (decimal, калории на 100 грама), Protein (decimal, протеин на 100 грама), Carbs (decimal, въглехидрати на 100 грама), Fats (decimal, мазнини на 100 грама), Barcode (string?, баркод на продукта за сканиране), CreatedAt (DateTime) и UpdatedAt (DateTime?).

Мутатор методи: *Update(string name, string? description, Guid categoryId, decimal calories, decimal protein, decimal carbs, decimal fat, string? barcode)* – обновява всички данни на продукта и задава UpdatedAt.

FoodCategory (таблица FoodCategories)

Моделът FoodCategory представлява категория храни. Полета: Id (Guid, първичен ключ), Name (string, име на категорията), Description (string?, описание) и CreatedAt (DateTime).

CalorieProgram (таблица CaloriePrograms)

Моделът CalorieProgram представлява дневните хранителни цели на потребителя за конкретна дата. Полета: ProgramID (Guid, първичен ключ), UserID (Guid, външен ключ към потребителя), CaloriesPerDay (int, общо целево количество калории за деня), CarbsPerDay (int, целево количество въглехидрати за деня), ProteinPerDay (int, целево количество протеини за деня), FatsPerDay (int, целево количество мазнини за деня), ProgramDate (DateTime, датата, за която се отнася хранителния план), CalorieProgramMeals (колекция от храни, включени в дневната програма).

CalorieProgramMeals (таблица CalorieProgramMeals)

Моделът CalorieProgramMeals свързва отделните храни (напр. закуска, обяд, вечеря) към конкретната дневна калорийна програма. Полета: MealID (int, първичен ключ), MealType (int, тип на храненето чрез Enum), CalorieProgramID (int, външен ключ, който свързва храненето към конкретен дневен план), MealsFood (колекция от конкретни храни и продукти, включени в специфичното хранене).

MealFood (таблица MealFoods)

Моделът MealFood свързва конкретни продукти и тяхното количество в рамките на дадено хранене. Полета: MealFoodId (int, първичен ключ), MealID (int, външен ключ към конкретно хранене), ProductID (int, външен ключ към избрания продукт), Weight (int, грамаж на консумирания продукт в това хранене).

Мутатор методи: *Update (int weight)* – обновява теглото (грамажа) на продукта в конкретното хранене.

Migration (таблица SystemMigrations)

Моделът SystemMigration следи изпълнените SQL миграции. Полета: MigrationID (Guid, първичен ключ, генерира се от базата с `gen_random_uuid()`), FileName (string, име на SQL файла, уникално) и CreatedAt (DateTime, кога е изпълнена миграцията).

4.10. Външни API endpoint-и

Комуникацията между мобилното приложение и сървъра се осъществява чрез RESTful API с JSON формат. Всички endpoint-и са групирани в шест контролера, всеки отговарящ за конкретен домейн от бизнес логиката.

AuthenticationController (api/Authentication)

POST api/Authentication/signin – Регистрация на нов потребител. Приема имейл и парола в JSON тяло. Генерира уникален Salt, хешира паролата с SHA-256 на 10000 итерации и създава нов запис в Users таблицата. Връща userId при успех или 400 Bad Request при дублиран имейл.

POST api/Authentication/login – Вход в системата. Приема имейл и парола. Извлича потребителя от базата по имейл, хешира подадената парола със съхранения Salt и сравнява с PasswordHash. Връща userId и token при успех или 401 Unauthorized при грешни данни.

POST api/Authentication/forgot-password – Заявка за код за възстановяване на парола. Приема имейл. Генерира 6-цифрен случаен код, записва го в PasswordResetCodes таблицата с ExpiresAt = сега + 5 минути и връща кода в отговора (симулация на имейл).

POST api/Authentication/verify-code – Потвърждение на код. Приема имейл и код. Проверява дали кодът съществува, не е изтекъл и не е използван. Връща 200 OK при валиден код или 400 Bad Request при невалиден.

POST api/Authentication/reset-password – Смяна на парола. Приема имейл и нова парола. Генерира нов Salt, хешира новата парола и обновява потребителския запис. Маркира кода като използван.

POST api/Authentication/biometric – Включване или изключване на биометрична автентикация. Приема userId и enable (bool). Обновява IsBiometricEnabled в Users таблицата.

CalendarController (api/Calendar)

GET api/Calendar/{userId} – Зареждане на календарни данни за конкретен месец. Приема userId като route параметър и year/month като query параметри. Изпраща CalendarDataQuery към MediatR, който извлича целевите калории (от TDEE изчислението) и записаните дневни калории от DailyCalories таблицата за дадения

месец. Връща обект с масив от дни, всеки съдържащ дата, изядени калории и калориен лимит. Използва се от `CalendarPage` за визуализация на калорийните пръстени.

DailyProgramController (api/DailyProgram)

GET `api/DailyProgram/{userId}` – Зареждане на дневния калориен план за конкретна дата. Приема `userId` и незадължителен `date` параметър (по подразбиране днешната дата). Изпраща `DailyProgramQuery` към `MediatR`. Ако за дадената дата все още няма създаден план, автоматично генерира нов чрез `CalorieProgramCommand` и след това го зарежда повторно. Връща обект с общи калории, макронутриенти и четирите ястия (закуска, обяд, вечеря, снакс). Използва се от `MainPage` при първоначално зареждане и при навигация между дните.

POST `api/DailyProgram/meal/food-to-meal` – Добавяне на храна към ястие. Приема `FoodToMealRequest` с полета: `MealFoodID` (за редакция на съществуващ запис или `null` за нов), `ProgramID` (идентификатор на дневния план), `MealType` (тип ястие), `MealID`, `ProductID` (идентификатор на продукта) и `Weight` (грамаж). Изпраща `FoodToMealCommand` към `MediatR`, който създава или обновява запис в `MealFoods` таблицата. Връща `mealId` при успех. Използва се от `FoodDetailPage` при натискане на „Добави към ястие“.

POST `api/DailyProgram/meal/delete-food` – Изтриване на храна от ястие. Приема `mealFoodId` (идентификатор на записа в `MealFoods`). Изпраща `FoodToMealDeleteCommand` към `MediatR`, който изтрива записа. Връща `removed=true` при успех. Използва се от `MealDetailPage` при натискане на иконката „X“.

GET `api/DailyProgram/meal/{mealId}/foods` – Зареждане на всички храни в конкретно ястие. Приема `mealId` като `route` параметър. Изпраща `MealFoodsQuery` към `MediatR`, който извлича всички записи от `MealFoods` таблицата за даденото ястие, заедно с данните за всеки продукт (име, калории, макронутриенти, грамаж). Използва се от `MealDetailPage` за показване на списъка с добавени храни.

DbConnectionController (api/DbConnection)

GET `api/DbConnection/db-connection` – Тестване на връзката с базата данни. Отваря и затваря връзка към PostgreSQL сървър чрез `AppDbContext`. Връща `connected=true` при успешна връзка или `StatusCode 500` с `error` и `innerException` при неуспех. Използва се за диагностика при разработка и за проверка дали Aiven Cloud сървърът е достъпен.

FoodController (api/Food)

GET `api/Food/foodcategory` – Списък с всички категории храни от `FoodCategories` таблицата.

GET `api/Food/category/{categoryId}` – Продукти по категория. Връща списък от `FoodProducts`, филтрирани по `CategoryId`, с калории и макронутриенти.

GET `api/Food/{productId}` – Търсене на продукт по уникален номер(`productId`). Филтрира `FoodProducts` по частично съвпадение на `productId`.

GET api/Food/create – Добавяне на храна към ястие. Приема FoodProductRequest с полета: ProductID (уникален идентификатор на продукта), ProductName (име на продукта), Discription(описание на продукта), Calories(калории на продукта), Weight(грамаж), Fats(мазнини), Protein(белтъци/протеини), Carbs(въглехидрати), Category(категория), Barcode(баркод, ако е бил добавен чрез баркод) . Изпраща FoodProductCommand към MediatR, който създава запис в FoodProducts таблицата. Връща съобщение “Продуктът е създаден успешно” при успех. Използва се за добавяне на продукти към базата данни.

GET api/Food/barcode/{barcode} – Търсене по баркод. Първо търси в локалната FoodProducts таблица. Ако не намери – прави заявка към Open Food Facts API (<https://world.openfoodfacts.org/api/v0/product/{barcode}.json>) за извличане на хранителна информация.

UserDetailsController (api/UserDetails)

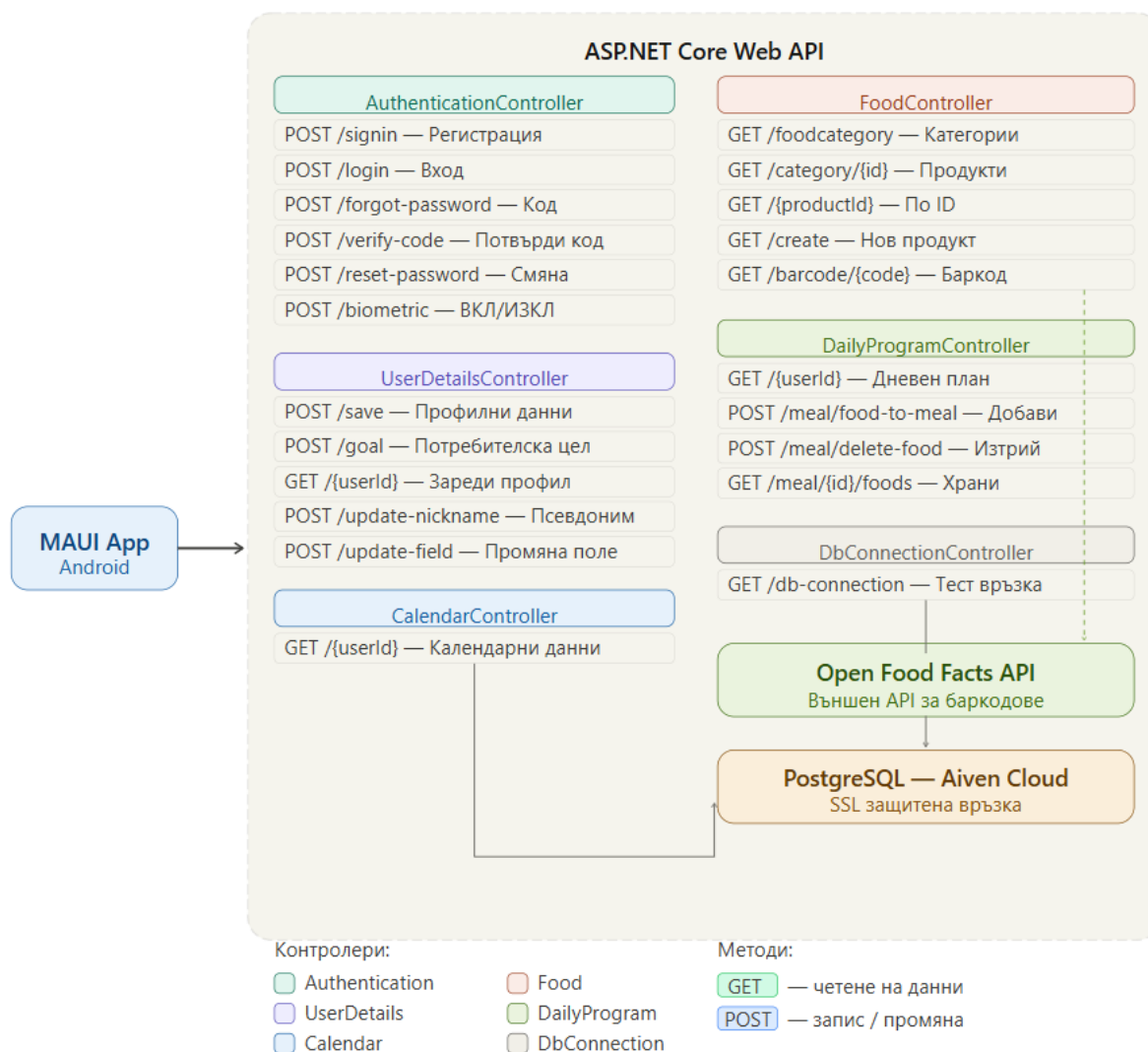
POST api/UserDetails/save – Запис на профилни данни след регистрация. Приема пълен обект с Nickname, Gender, DateOfBirth, HeightCm/Ft, WeightKg/Lbs, ActivityLevel, CurrentGoal, TargetWeightKg/Lbs. Проверява дали вече съществува запис за потребителя – ако да, обновява го; ако не – създава нов. Автоматично конвертира между мерни единици.

POST api/UserDetails/goal – Запис на потребителска цел. Приема GoalType, TargetWeightKg/Lbs, StartDate, EndDate. Същата логика за проверка на съществуващ запис.

GET api/UserDetails/{userId} – Зареждане на профилни данни. Връща всички полета от UserDetails и текущата цел от UserGoals в JSON формат. Използва се от ProfilePage при отваряне.

POST api/UserDetails/update-nickname – Промяна на псевдоним. Приема userId и новия Nickname. Търси записа по UserId и извиква UpdateNickname() метода.

POST api/UserDetails/update-field – Промяна на отделно поле. Приема userId, fieldName (string) и value (string). Търси записа по UserId и извиква UpdateField() метода, който парсва стойността и автоматично конвертира мерни единици.



Фиг. 12 – Схема на външни API endpoints

4.11. Реализация на Repository Pattern

Репозитори шаблонът е реализиран чрез единен генеричен интерфейс IRepository, който работи с всеки модел без необходимост от отделни репозитори класове за всяка таблица. Методите Set<T>() и SetNoTracking<T>() приемат params string[] параметър за Include на навигационни свойства, което позволява гъвкаво зареждане на свързани данни.

Методът Save<T>(entity, exists) автоматично определя дали обектът трябва да бъде добавен или обновен в базата данни, опростявайки логиката в обработчиците.

4.12. Система за миграции

Вместо стандартните EF Core миграции, проектът използва собствена система за миграции. SQL файловете се поставят в папката DataLayer/Migrations/ с номерирана

номенклатура(например 20260405_CREATE/ALTER/UPDATE_ИметоНаТаблицата.sql). При стартиране на API, MigrationRunner извлича всички SQL файлове от embedded resources, проверява кои от тях вече са изпълнени (чрез таблицата SystemMigrations) и изпълнява само новите в транзакция.

Този подход дава пълен контрол над SQL заявките, позволява по-сложни миграции (ALTER TABLE, CREATE INDEX, INSERT начални данни) и е по-предвидим при екипна работа.

4.13. Калориен календар

Календарът визуализира калорийните данни по месеци. API ендпойнтът GET /api/dailyprogram/calendar/{userId} приема параметри year и month и връща масив от дни, като за всеки ден се предоставят целта (CaloriesGoal), изядените калории (CaloriesEaten) и наличието на план (HasProgram).

CalendarViewModel изгражда, за клиента, мрежа от 42 клетки (6 седмици), като попълва дните от предишния и следващия месец за визуална пълнота. За всеки ден с данни се изчислява прогрес (CaloriesProgress) и индикатор за превишение (IsOverTarget). Визуално днешният ден е маркиран със син фон, а другите дни от месеца или годината се маркират само със син пръстен, но при избор на тях от потребителя. Изядените калории се показват с малко по-голям пръстен около всеки конкретен ден - всички дни от създаването на профила до днешния ден. Калорийните пръстени се изобразяват в два цвята: зелен при спазване на целта и червен при превишение на целта.

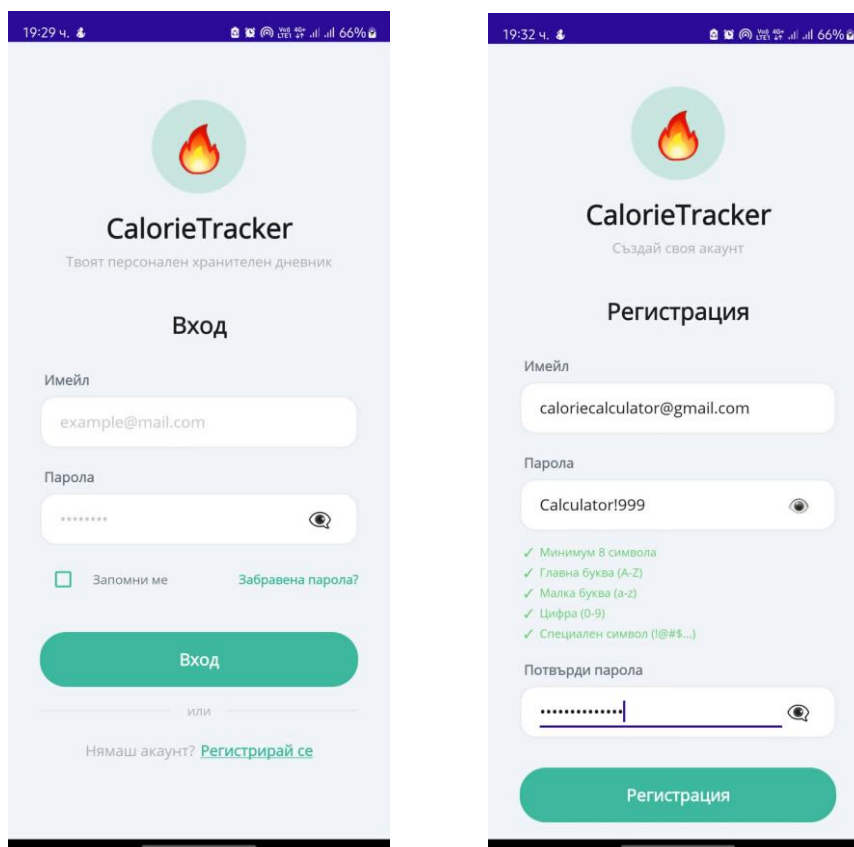
5. Потребителско ръководство

5.1. Регистрация и вход

При първоначално стартиране на приложението потребителят вижда страницата за вход (LoginPage). От тук може да се автентикира с имейл и парола или да премине към регистрация.

Регистрацията изисква въвеждане на имейл адрес и парола с потвърждение. Имейл адресът се валидира за коректен формат и се изисква да съдържа само малки букви – при опит за въвеждане на главна буква системата показва съобщение „Имейлът трябва да съдържа само малки букви“ вместо автоматично преобразуване, за да е наясно потребителят с точния имейл, който използва. Паролата преминава през реално-времева валидация спрямо международния стандарт за информационна сигурност. При въвеждане на всеки символ потребителят вижда динамичен индикатор (зелена отметка) за всяко от петте изисквания: минимум 8 символа, поне една главна буква (A-Z), поне една малка буква (a-z), поне една цифра (0-9) и поне един специален символ (!@#\$...). Бутонът за регистрация се активира едва когато всички изисквания са изпълнени и двете пароли съвпадат. Паролата също може да бъде показана/скрита чрез иконка за видимост (око). След успешна регистрация потребителят е насочен към страницата за настройка на профила.

При вход потребителят въвежда имейл и парола. Страницата предлага опция „Запомни ме“, която запазва имейл адреса за следващо влизане, както и линк „Забравена парола?“ за възстановяване на парола. Ако потребителят е активирал биометричния вход от профилната си страница, след успешна проверка на имейл и парола (първи фактор) Android показва системния диалог за пръстов отпечатък (втори фактор). Едва след успешно биометрично потвърждение потребителят влиза в приложението.



Фиг. 13 - Графично представяне на страниците за вход/регистрация на мобилно устройство

5.2. Забравена парола

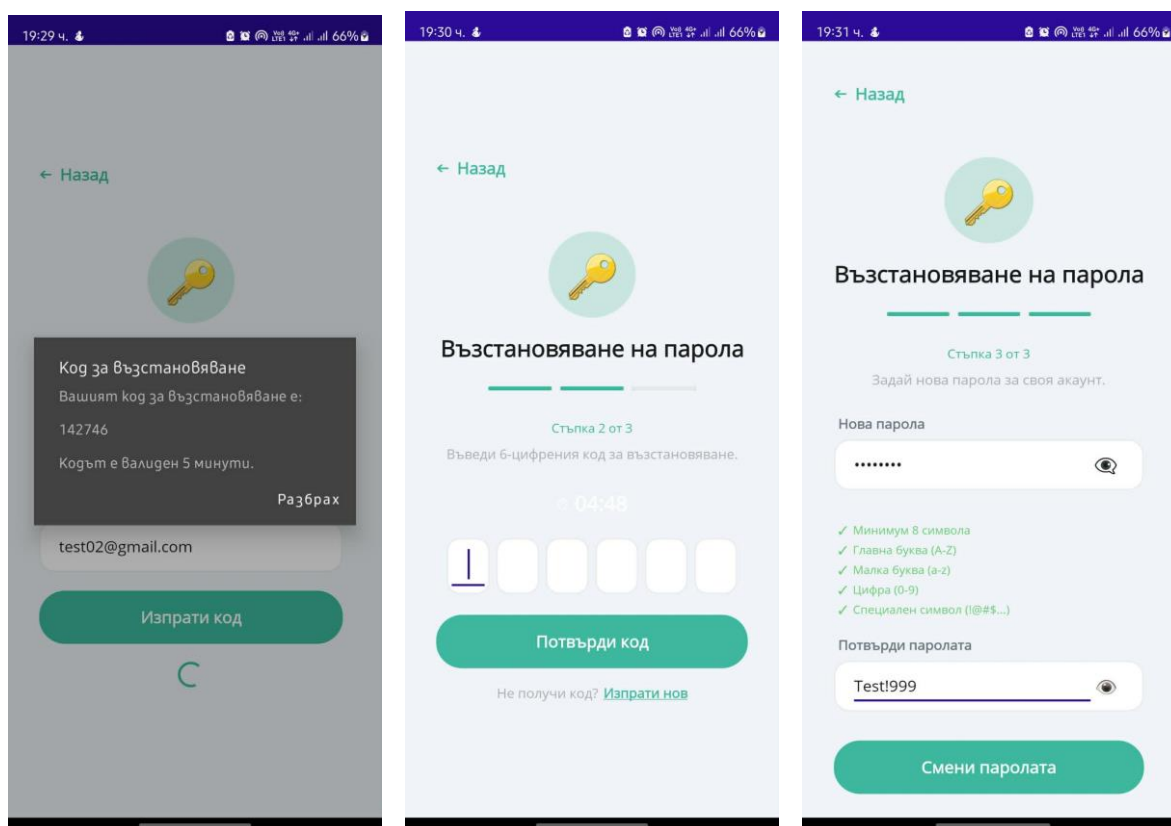
От страницата за вход потребителят натиска „Забравена парола?“ и преминава през тристъпков wizard (ForgotPasswordPage). В горната част се показва прогрес индикатор от три сегмента, оцветени в зелено за преминатите стъпки и сиво за предстоящите.

На първата стъпка потребителят въвежда имейла, с който се е регистрирал, и натиска „Изпрати код“. API генерира 6-цифрен код за възстановяване, който се показва чрез DisplayAlert диалог (симулация на имейл изпращане). Кодът се съхранява в таблицата PasswordResetCodes в базата данни с еднократна употреба и изтичане след 5 минути.

На втората стъпка потребителят въвежда получения 6-цифрен код в шест отделни полета. Полетата имат автоматичен фокус – при въвеждане на цифра фокусът преминава към следващото поле. Над полетата се показва обратен брояч от 5 минути, като текстът на таймера става червен при изтичане. Ако времето изтече, потребителят може да поиска

нов код чрез линка „Изпрати нов“. При натискане на „Потвърди код“ API проверява дали кодът е валиден и не е изтекъл.

На третата стъпка потребителят задава нова парола. Паролата трябва да покрива всички ISO/IEC 27001 изисквания, визуализирани с динамичен зелен индикатор (както при регистрацията): минимум 8 символа, главна буква, малка буква, цифра и специален символ. Паролата може да бъде показана/скрита чрез иконката за видимост. Потребителят потвърждава новата парола във второ поле. След успешна смяна на паролата потребителят е насочен обратно към страницата за вход.



Фиг. 14 - Графично представяне на страниците за забравена парола на мобилно устройство

5.3. Настройка на профила

След успешна регистрация потребителят преминава през 8-стъпков wizard за попълване на детайлите за профила (ProfileSetupPage). Всяка стъпка е задължителна – потребителят не може да продължи напред без да попълни полето. В горната част се показва прогрес

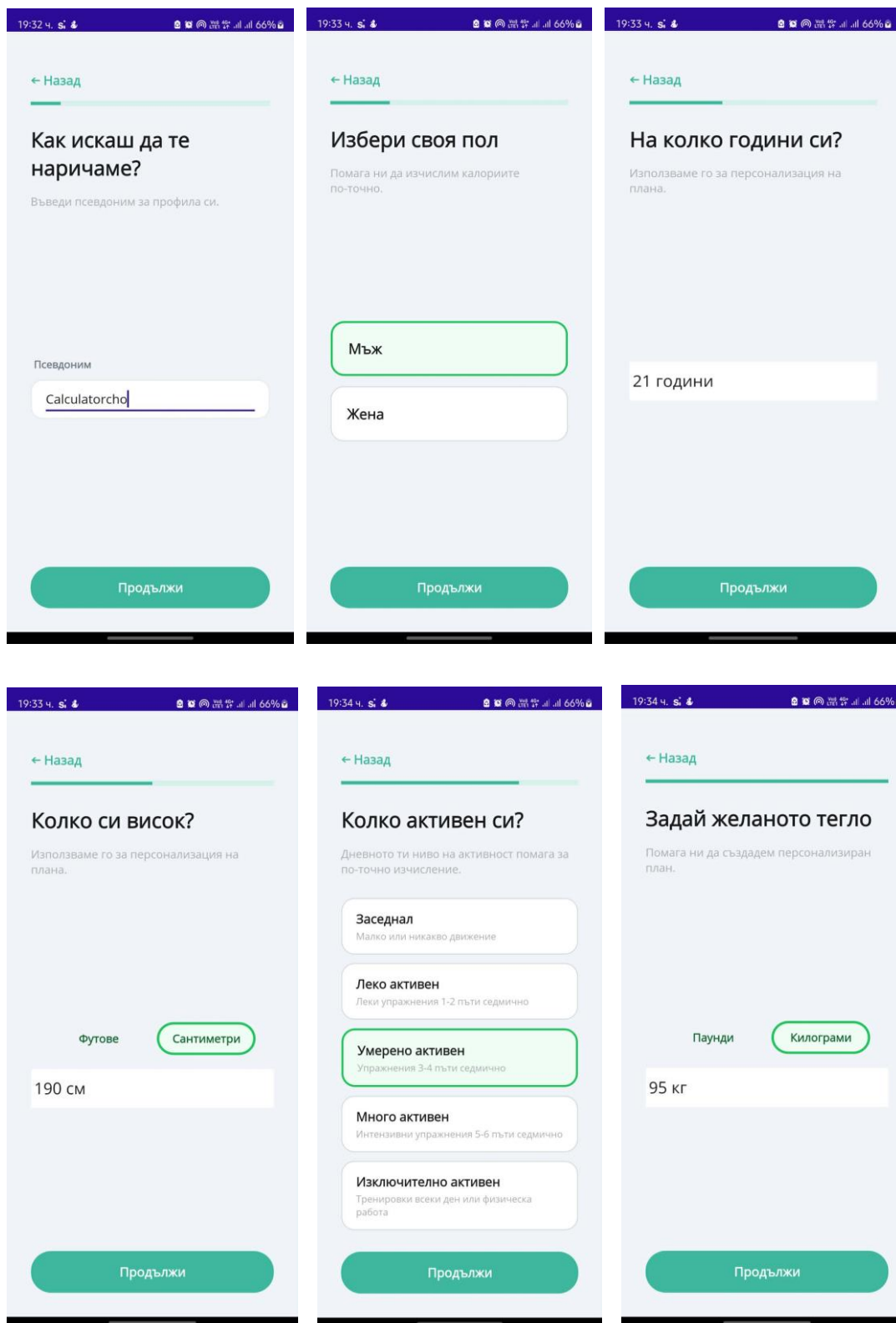
бар, който визуализира на коя стъпка от осемте се намира потребителят, както и бутон „Назад“ за връщане към предишната стъпка.

Стъпките са следните:

- Стъпка 1 – Псевдоним (минимум 2 символа)
- Стъпка 2 – Пол (Мъж или Жена, избира се чрез натискане на карта)
- Стъпка 3 – Възраст (избира се чрез Picker компонент)
- Стъпка 4 – Ръст, с toggle бутон за избор между сантиметри и футове (при избор на една мерна единица, другата се изчислява автоматично и се записва в базата данни)
- Стъпка 5 – Тегло, с toggle бутон за избор между килограми и паундове (същата автоматична конверсия, както при ръста)
- Стъпка 6 – Ниво на активност (5 опции от „Заседнал“ до „Изключително активен“, всяка с кратко описание)
- Стъпка 7 – Цел (Загуба на тегло, Задържане на теглото, Качване на тегло, Качване на мускулна маса)
- Стъпка 8 – Желано тегло, с toggle бутон за кг/паундове

На последната стъпка се извършва логическа валидация спрямо избраната цел: при цел „Загуба на тегло“ желаното тегло трябва да е по-малко от текущото, при „Качване на тегло“ или „Качване на мускулна маса“ – по-голямо, а при „Задържане на теглото“ – равно на текущото. При несъответствие се показва съобщение за грешка.

Въз основа на тези данни системата автоматично изчислява дневния калориен план по формулата на Mifflin-St Jeor, с корекция спрямо нивото на активност и целта, както и разпределение на макронутриентите (протеин, въглехидрати, мазнини).



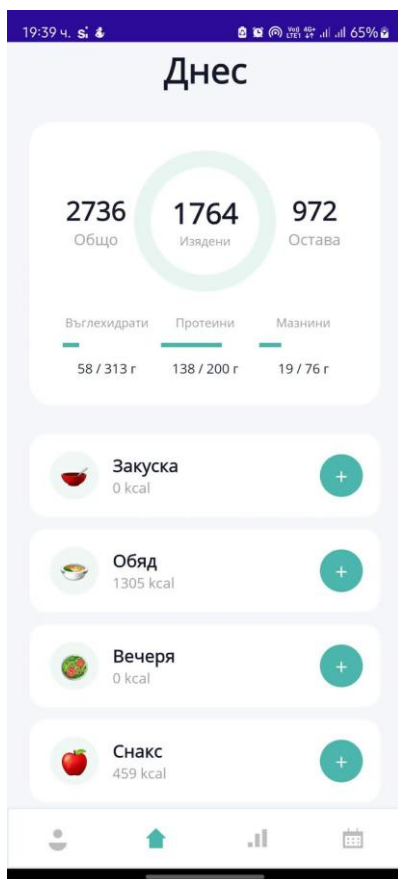
Фиг. 15 - Графично представяне на страниците за попълване детайли на профила на мобилно устройство

5.4. Главна страница

Главната страница (MainPage) показва дневния калориен план с визуален тракер. В горната част се вижда общият брой калории, изядените и оставащите. Под тракера са представени три прогрес бара за макронутриенти: протеин, въглехидрати и мазнини.

В долната част се показват четирите ястия: закуска, обяд, вечеря и снакс. За всяко ястие се вижда текущият брой калории. Натискането на бутона „+“ отваря страницата с категории храни за добавяне на продукт. Натискането на самото ястие отваря детайлния изглед с добавените храни.

Стрелките за навигация по дати позволяват преглед на предишни или следващи дневни планове. Ако е наличен предишен план, се показва стрелка наляво; ако е наличен следващ – стрелка надясно.



Фиг. 16 - Графично представяне на основната страница на мобилно устройство

5.5. Добавяне на храна

Процесът на добавяне на храна преминава през следните стъпки:

1. От главния екран потребителят натиска „+“ на избрано ястие.
2. Отваря се списък с категории храни. Търсачката филтрира категориите в реално време.
3. При избор на категория се показват продуктите в нея с калории и макроси.
4. При избор на продукт се отваря детайлна страница с поле за грамаж.
5. Промяната на грамажа автоматично преизчислява калориите и макросите в реално време.
6. Бутонът „Добави към ястие“ записва храната и връща потребителя към главния екран.

21:04 ч. 51%

← Нов продукт

Име на продукт

Агнешки ребра

Описание

печени

Категория

Месо и месни продукти

Грамаж (г)

100

Хранителни стойности (на 100г)

Калории (kcal) Протеин (г)

300 24

Въглехидрати (г) Мазнини (г)

0 27

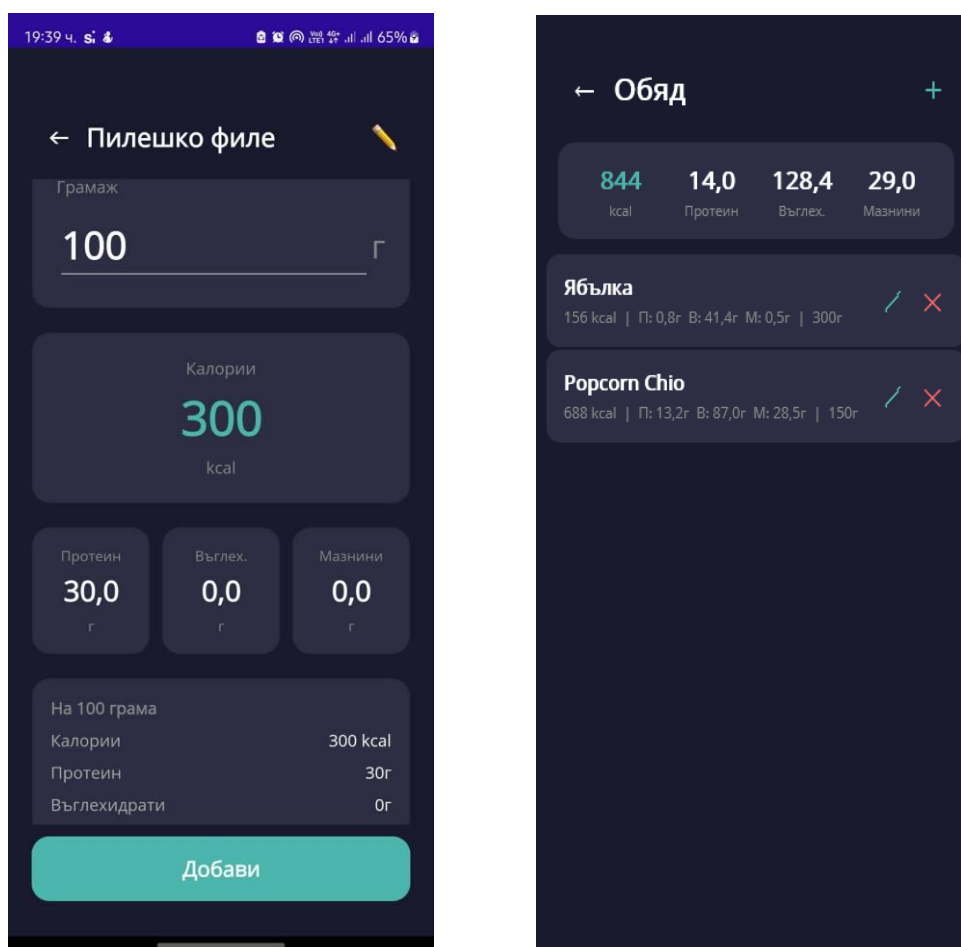
Запази продукт

Фиг. 17 - Графично представяне на страницата за добавяне на продукт на мобилно устройство

5.6. Управление на ястие

При натискане на ястие от главния екран се отваря MealDetailPage, където потребителят вижда всички добавени храни с техните калории, макронутриенти и грамаж. За всяка храна са налични два бутона:

- Икона молив – отваря детайлната страница на продукта с текущия грамаж, позволявайки промяна.
- Икона „X“ – незабавно изтрива храната от ястието без потвърждение.

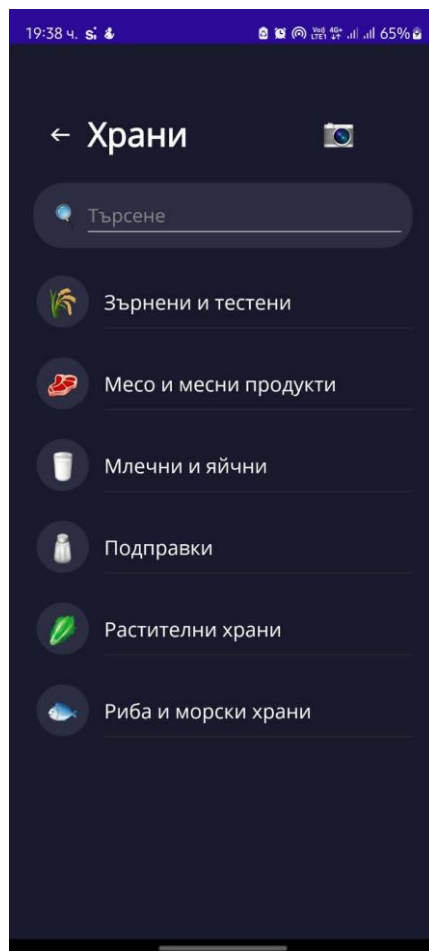


Фиг. 18 - Графично представяне страницата за управление на ястие на мобилно устройство

5.7. Създаване на нов продукт

Потребителят може да създаде нов хранителен продукт чрез бутона „+“ в горния десен ъгъл на страницата с категории или продукти. Формулярът изисква: име на продукта,

описание (незадължително), категория, грамаж, калории, протеин, въглехидрати и мазнини. Същият формуляр се използва и за редактиране на съществуващ продукт.

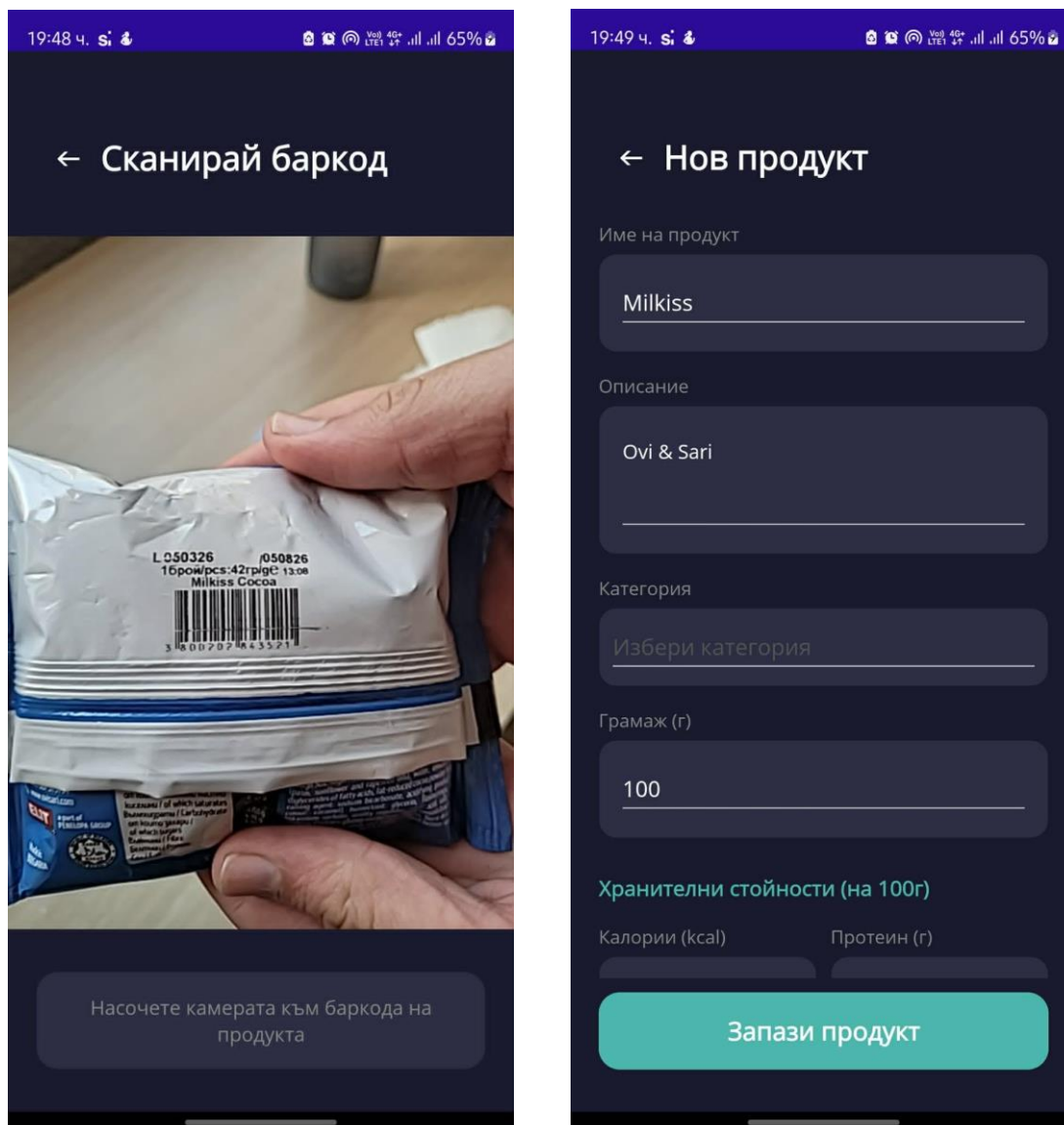


Фиг. 19 - Графично представяне на страницата с всички категории храни на мобилно устройство

5.8. Сканиране на баркод

Иконата на камера в страницата с категории активира баркод скенера. Потребителят насочва камерата към баркода на продукта. При успешно разпознаване:

- Ако продуктът е в базата данни – директно отваря страницата за добавяне.
- Ако е намерен в Open Food Facts – отваря формуляра за създаване с попълнени данни.
- Ако не е намерен никъде – отваря празен формуляр с прикачен баркод.



Фиг. 20 - Графично представяне на страниците за сканиране на баркод и добавяне на сканирания продукт на мобилно устройство

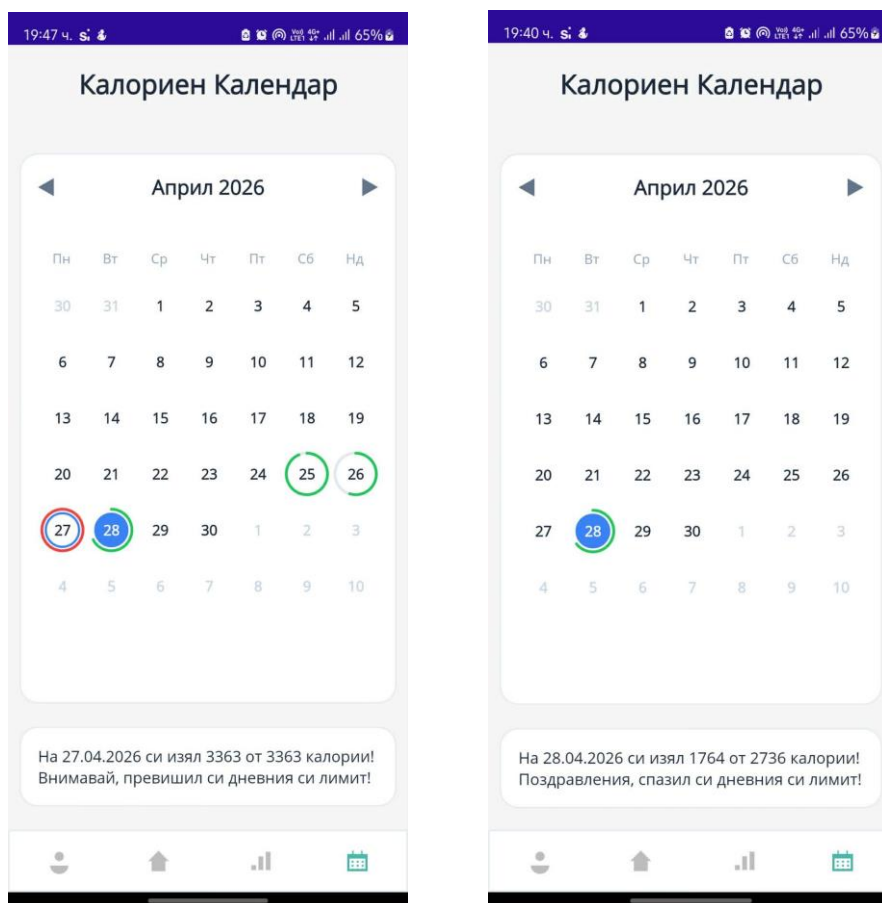
5.9. Калориен календар

Разделът „Календар“ (CalendarPage) показва пълен месечен изглед с навигационни стрелки (◀ ▶) за преместване между месеците. Имената на месеците и дните са на български език. Днешният ден е маркиран със запълнен син кръг. При натискане на друг ден, той се огранчава с тънка синя рамка без запълване, за да се разграничава от днешния.

Около всеки ден с записани калории се показва калориен пръстен – зелена дъга, която расте по часовниковата стрелка, пропорционално на процента изядени калории спрямо

дневния лимит. Зад дъгата се вижда сив фонен кръг, представящ 100% от лимита. При 30% прогрес се вижда малка дъга, при 50% – дясната половина, при 80% – почти пълен кръг. Ако калориите надвишат дневния лимит (над 100%), пръстенът се запълва изцяло и става червен. Конверсията на прогреса в дъга се извършва чрез специализирания конвертор ProgressToArcPathConverter, който генерира PathGeometry с ArcSegment за съответния ъгъл.

При натискане на конкретен ден под календара се появява информационен панел със съобщение: „На [дата] си изял [X] от [Y] калории!“ последвано от „Поздравления, спазил си дневния си лимит!“ при спазена цел или „Внимавай, превишил си дневния си лимит!“ при превишение. За бъдещи дати се показва „На [дата] все още няма записани данни!“, а за дни без записи – „На [дата] няма записани калории!“.



Фиг. 21 - Графично представяне на страницата „Календар“ на мобилно устройство

5.10. Профилна страница

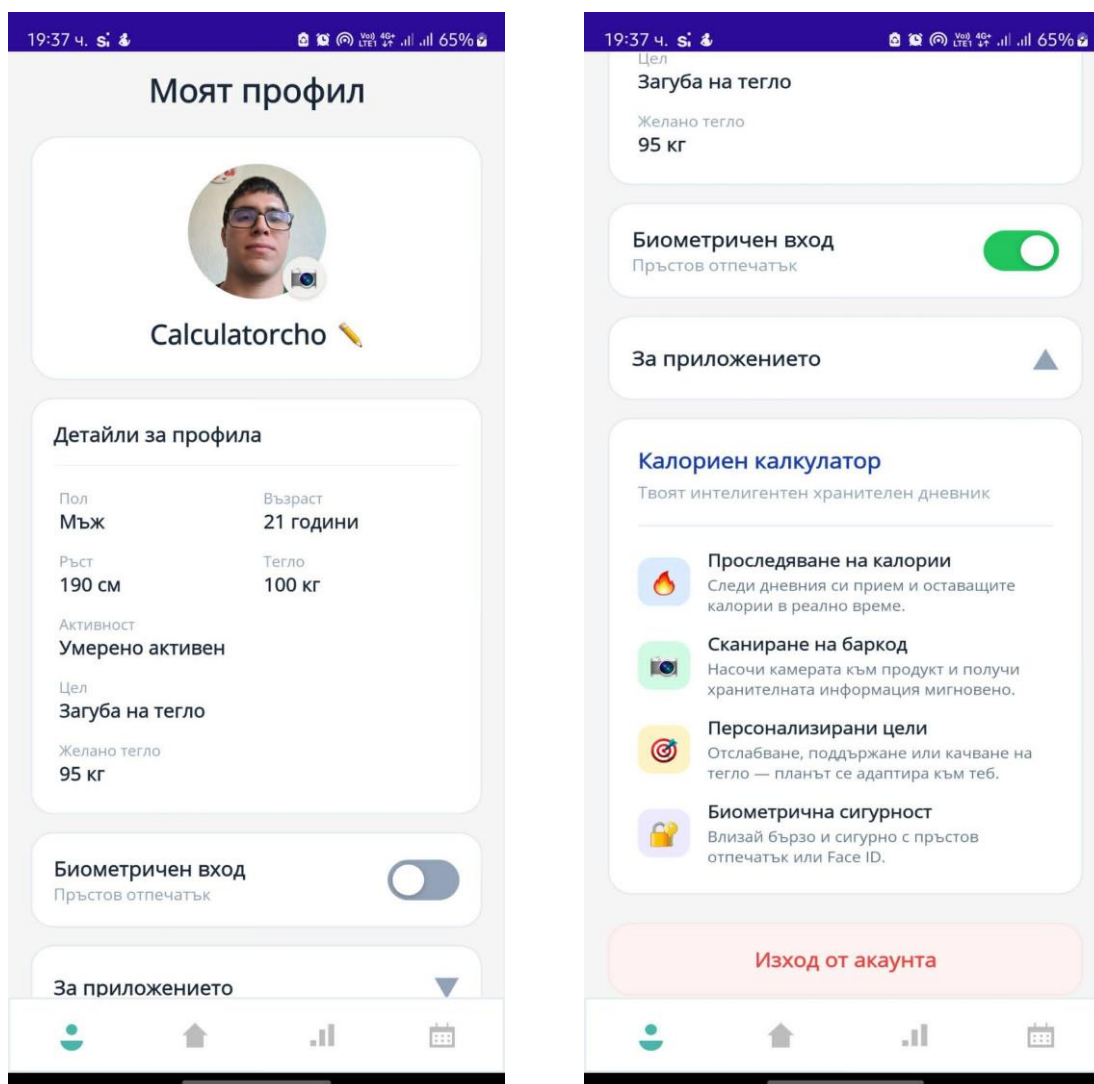
Профилната страница (ProfilePage) е централната точка за управление на потребителския акаунт. В горната част се показва аватар – цветен кръг (син за мъже, розов за жени) с първата буква от псевдонима. Ако потребителят е направил профилна снимка (селфи), тя замества цветния кръг. В долния десен ъгъл на аватара е позиционирана камера иконка. При натискане върху аватара се показва ActionSheet с две опции: „Направи селфи“ и „Премахни снимката“. При избор на „Направи селфи“ се отваря камерата на устройството за заснемане на снимка в реално време. Снимката се запазва локално с уникален идентификатор за всеки потребител – различни акаунти на едно устройство имат отделни профилни снимки. Снимката се запазва и след излизане от акаунта.

Под аватара е показан псевдонимът с иконка за редактиране. При натискане псевдонимът се заменя с поле за въвеждане и два бутона – „Запази“ и „Откажи“. Псевдонимът трябва да е поне 2 символа. Промяната се записва веднага в базата данни чрез API заявка.

Секцията „Детайли за профила“ показва всички телесни данни в два вида: нередактируеми (пол, възраст) и редактируеми (ръст, тегло, активност, цел, желано тегло). При натискане на числово поле (ръст, тегло, желано тегло) изскача промпт диалог с клавиатура за числа. При натискане на „Активност“ или „Цел“ изскача ActionSheet със списък от опции за избор. Данните се показват в мерната единица, избрана при регистрация – ако потребителят е избрал футове и паундове, те се визуализират съответно. При промяна на тегло или желано тегло се извършва валидация спрямо текущата цел, за да не се допускат несъответствия. Всяка промяна се записва автоматично в базата данни, като API-то конвертира и алтернативната мерна единица.

По-долу е toggle бутонът за биометричен вход с визуален индикатор (сив при изключен, зелен при включен) и текст „Пръстов отпечатък“. Следва секцията „За приложението“ с разгъваемо описание на функционалностите на CalorieCalculator (проследяване на калории, сканиране на баркод, персонализирани цели, биометрична сигурност). Накрая е бутонът „Изход от акаунта“ в червен цвят, който пита за потвърждение „Сигурни ли сте, че искате да излезете от профила си?“ преди да изчисти сесията и да насочи

потребителя към страницата за вход. При излизане се запазват настройките за биометрия, запомнен имейл и профилна снимка.



Фиг. 22 - Графично представяне на профил страница на мобилно устройство

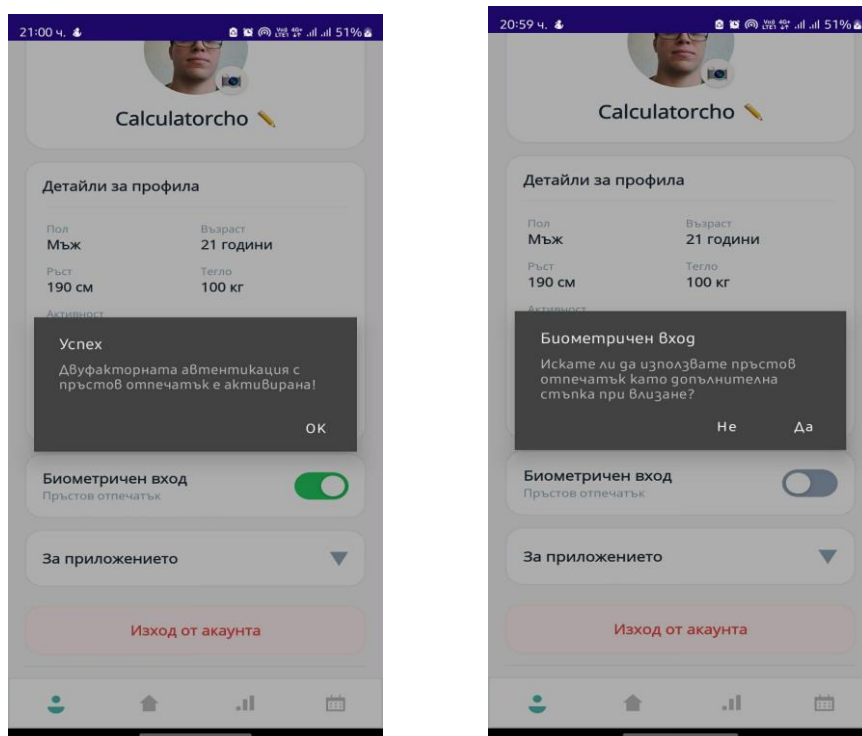
5.11. Биометричен вход

За да активира биометричния вход, потребителят отива в профилната страница и натиска toggle бутона „Биометричен вход“. Системата показва диалог „Искате ли да използвате пръстов отпечатък като допълнителна стъпка при влизане?“ с бутони „Да“ и „Не“. При натискане на „Да“ Android показва системния диалог за пръстов отпечатък, използвайки хардуерния биометричен сензор на устройството чрез библиотеката Plugin.Fingerprint. При успешно сканиране toggle бутонът преминава в зелен цвят и се показва съобщение

„Двуфакторната автентикация с пръстов отпечатък е активирана!“. Настройката се записва както локално в Preferences, така и в базата данни (колона IsBiometricEnabled в таблицата Users).

При следващо влизане процесът е двуфакторен: потребителят въвежда имейл и парола и натиска „Вход“ (първи фактор – нещо, което знае). API проверява данните и при успех системата автоматично проверява дали биометрията е включена за този потребител. Ако да – Android показва системния fingerprint диалог (втори фактор – нещо, което притежава). Потребителят поставя пръст върху биометричния сензор и при успешно разпознаване влиза в приложението. При неуспешно сканиране се показва съобщение „Биометричната автентикация е неуспешна. Опитайте отново.“ и потребителят може да опита повторно.

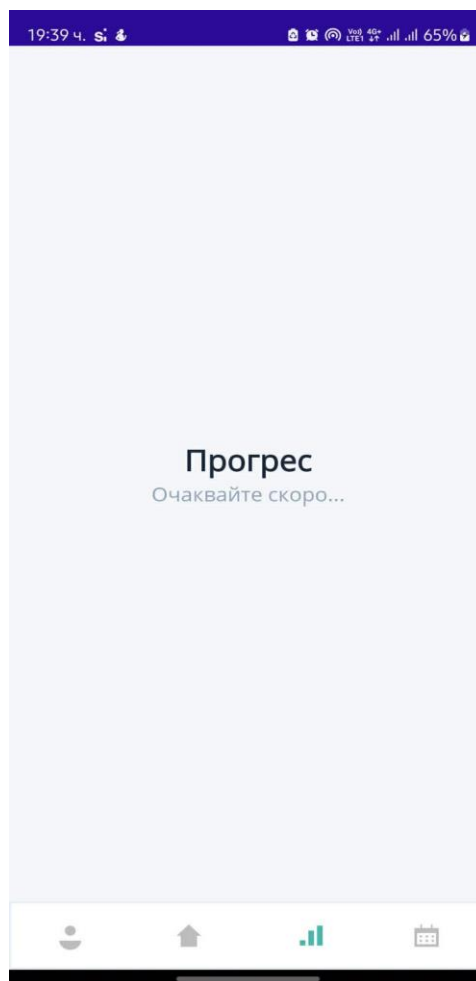
Биометрията може да бъде изключена по всяко време от профилната страница чрез повторно натискане на toggle бутона, с потвърждение. Настройката е индивидуална за всеки потребител (per-user) – различни акаунти на едно устройство имат отделни настройки. При нова регистрация биометрията е винаги изключена по подразбиране и се активира само по желание на потребителя.



Фиг. 23 - Графично представяне на работата на биометрията на мобилно устройство

5.12. Прогрес

Разделът „Прогрес“ (ProgressPage) е предназначен за визуализация на дългосрочния напредък на потребителя. В текущата версия на приложението страницата показва съобщение „Очаквайте скоро...“, като планираните функционалности включват: графики за промяна на теглото във времето, седмични и месечни статистики и графики за калорийния прием, както и постижения за мотивация на потребителите. Страницата е достъпна чрез долната навигационна лента (NavBar) от всички основни страници на приложението и е подготвена за бъдещо разширение с данни от таблицата DailyCalories.



Фиг. 24 - Графично представяне на страницата „Прогрес“ на мобилно устройство

6. Заключение

Настоящият проект представя CalorieCalculator – мобилно приложение за проследяване на калориен прием, което обхваща пълния цикъл от регистрация на потребителя до визуализация на напредъка. Приложението е разработено с използване на съвременни технологии и архитектурни подходи, осигуряващи мащабируемост, поддръжка и разширяемост.

6.1. Постигнати резултати

В рамките на проекта бяха постигнати следните резултати:

- Разработено мобилно приложение с интуитивен потребителски интерфейс, позволяващ бързо и удобно проследяване на калорийния прием.
- Изградена сървърна архитектура с ясно разделение на отговорностите чрез слоеста архитектура, Repository Pattern и CQRS.
- Реализирана сигурна система за автентикация с хеширане на пароли.
- Интегриран баркод скенер с връзка към Open Food Facts API за бързо добавяне на продукти.
- Реализиран калориен календар за месечен преглед на хранителните данни.
- Използвани четири хардуерни функции на мобилното устройство: камера за баркод скенер (ZXing.Net.Maui), камера за профилна снимка/селфи (MAUI MediaPicker), биометричен сензор за двуфакторна автентикация с пръстов отпечатък (Plugin.Fingerprint) и мрежова свързаност (Wi-Fi/мобилни данни) за комуникация с API сървъра.

6.2. Бъдещо развитие

Приложението може да бъде разширено в следните направления:

- GPS проследяване на физическа активност за автоматично изчисляване на изгорени калории.
- Графики и статистики за дългосрочен анализ на напредъка.
- Интеграция с Bluetooth устройства (фитнес гривни, кантари) за автоматично синхронизиране на данни.

- Социални функции – споделяне на рецепти и хранителни планове между потребители.
- Поддръжка на iOS платформа чрез .NET MAUI.

В заключение, CalorieCalculator представлява функционално и технически издържано решение, което отговаря на поставените цели и е готово за реална употреба. Модулната архитектура и чистият код осигуряват лесно бъдещо разширяване и поддръжка.

7. Приложение – ключови фрагменти от кода

7.1. IRepository интерфейс

Абстракция за управление на Entity към базата данни. Чрез тази абстракция и имплементирането на интерфейса от [Repository.cs](#) се постига централизирано и абстрактно разбиране на необходимите действия изпълнявани върху различните Entity-та. Това включва:

Add - добавяне на Entity към съответната таблица

Update - обновяване на вече съществуващо Entity

Delete - изтриване на Entity от съответната таблица

Save - използва вече създадените методи Add и Update като определя кой от тях да изпълни на базата на флаг който се подава дали Entity-то съществува или не

Set - извличане на Entity-та с опция за промяна върху полетата.

SetNoTracking - извличане на Entity-та без опция за промяна върху полетата

Set и SetNoTracking с параметри - разликата между другите два е че може да се направи връзка по външен ключ и да се изкара информация за връзките на Entity-то

Query - основно се ползва за извличане на всички Entity-та от таблица и прилагане на множество филтри

Find - за подаване на функции и търсене на конкретни Entity-та след тяхното изпълнение

ExecuteSql - възможност за изпълняване на sql query под формата на string, по този начин могат да се викат динамично процедури, функции или view таблици

BeginTransaction - започва така наречената “транзакция”, която се използва при промяна на Entity с един или повече от посочените методи горе. Ако възникне грешка при вече отворена “транзакция” данните се не се запзват и нищо не се променя по таблиците. Промяната се случва след като “транзакцията” бъде комитната.

SaveChanges - запазва промените като предпазва от презаписване на данни и загуба на такива. Това се прави след модификация на Entity и преди комит на “транзакцията”.

```
public interface IRepository
{
    Task Add<T>(T entity) where T : class;

    void Update<T>(T entity) where T : class;

    void Delete<T>(T entity) where T : class;

    void Save<T>(T entity, bool exists) where T : class;
```

```

IQueryable<T> Set<T>() where T : class;
IQueryable<T> Set<T>(params string[] includes) where T : class;
IQueryable<T> SetNoTracking<T>() where T : class;
IQueryable<T> SetNoTracking<T>(params string[] includes) where T : class;
IQueryable<T> Query<T>() where T : class;
Task<List<T>> Find<T>(Expression<Func<T, bool>> predicate) where T : class;
Task<int> ExecuteSql(string sql);
Task<IDbContextTransaction> BeginTransaction();
Task<int> SaveChanges();
}

```

7.2. IAuthorization интерфейс

Абстракция за управление на ауторизацията в мобилното приложение:

```

public interface IAuthorization
{
    Task<bool> SignIn(string email, string password);
    Task<bool> LogIn(string email, string password);
    Task<string?> GeneratePasswordResetCode(string email);
    Task<bool> VerifyPasswordResetCode(string email, string code);
    Task<bool> ResetPassword(string email, string code, string newPassword);
    string HashPassword(string email, string salt);
    string GenerateSalt();
}

```

7.3. MigrationRunner

Това е service за автоматично изпълнение на PostgreSQL миграции, той има за цел да пази история на всички скриптове изпълнявани върху базата данни, новосъздадени всички скриптове се изпълняват по време на компилиране и всички изпълнени миграции се пазят в базата с id, fileName и time stamp. Таблицата SystemMigrations се създава преди изпълнението на първата миграция в система от:

```
private async Task CreateSystemMigrationsIfNotExists
```

```

public async Task RunMigrationsAsync()
{
    await CreateSystemMigrationsIfNotExists();
}

```

```

var assembly = Assembly.GetExecutingAssembly();
var resourceNames = assembly.GetManifestResourceNames()
    .Where(name => name.EndsWith(".sql"))
    .OrderBy(name => name)
    .ToList();

foreach (var resourceName in resourceNames)
{
    var fileName = resourceName.Substring(
        resourceName.LastIndexOf('.', resourceName.LastIndexOf('.') - 1) + 1);

    var migration = _repository.SetNoTracking<Migration>()
        .FirstOrDefault(m => m.FileName == fileName);

    if (migration != null)
    {
        continue;
    }

    using var stream = assembly.GetManifestResourceStream(resourceName);
    using var reader = new StreamReader(stream!);
    var sql = await reader.ReadToEndAsync();

    await using var transaction = await _repository.BeginTransaction();
    try
    {
        await _repository.ExecuteSql(sql);

        await _repository.Add(new Migration
        {
            MigrationID = Guid.NewGuid(),
            FileName = fileName,
            CreatedAt = DateTime.UtcNow
        });

        await _repository.SaveChanges();

        await transaction.CommitAsync();
    }
    catch (Exception ex)
    {
        await transaction.RollbackAsync();
        Console.WriteLine($"Failed: {fileName} - {ex.Message}");
        throw;
    }
}

private async Task CreateSystemMigrationsIfNotExists()
{
    var sql = @"
CREATE TABLE IF NOT EXISTS ""SystemMigrations"" (
    ""MigrationID"" uuid DEFAULT gen_random_uuid() PRIMARY KEY,
    ""FileName"" VARCHAR(255) NOT NULL UNIQUE,
    ""CreatedAt"" TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);";

    await _repository.ExecuteSql(sql);
}

```

7.4. Пример за SQL миграция

Създаване на таблицата FoodProducts:

```
CREATE TABLE IF NOT EXISTS "FoodCategories" (  
    "CategoryID" INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    "Name" VARCHAR (255) NOT NULL UNIQUE  
);  
  
CREATE TABLE IF NOT EXISTS "FoodProducts" (  
    "ProductID" INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    "Name" VARCHAR (255) NOT NULL,  
    "Description" VARCHAR (255),  
    "CategoryID" INT NOT NULL,  
    "Calories" INT NOT NULL,  
    "Fats" DECIMAL NOT NULL,  
    "Protein" DECIMAL NOT NULL,  
    "Carbs" DECIMAL NOT NULL,  
    "Weight" INT NOT NULL,  
    "CreatedAt" TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_category FOREIGN KEY ("CategoryID")  
    REFERENCES "FoodCategories"("CategoryID")  
);
```

Промяна на таблицата UsersLoginData:

```
ALTER TABLE "Users" ALTER COLUMN "Id" DROP IDENTITY IF EXISTS;  
ALTER TABLE "Users" DROP CONSTRAINT IF EXISTS "Users_pkey" CASCADE;  
  
ALTER TABLE "Users"  
    ALTER COLUMN "Id" TYPE UUID USING gen_random_uuid();  
  
ALTER TABLE "Users" ALTER COLUMN "Id" SET DEFAULT gen_random_uuid();  
ALTER TABLE "Users" ADD PRIMARY KEY ("Id");  
  
DROP INDEX IF EXISTS "User_Id";  
CREATE UNIQUE INDEX IF NOT EXISTS "IX_Users_Email" ON "Users" ("Email");
```

Модифициране на таблицата FoodProducts:

```
UPDATE public."FoodCategories"  
SET "Name" = 'Месо и месни продукти'  
WHERE "Name" = 'Месо и местни продукти';
```


7.5. ApiService (MAUI)

Централизиран клас за HTTP комуникация:

```
public class ApiService
{
    private readonly HttpClient _httpClient;

    public async Task<T?> GetAsync<T>(string endpoint)
    {
        var response = await _httpClient.GetAsync(endpoint);
        response.EnsureSuccessStatusCode();
        return await response.Content
            .ReadFromJsonAsync<T>();
    }

    public async Task<T?> PostAsync<T>(string endpoint,
        object data)
    {
        var response = await _httpClient
            .PostAsJsonAsync(endpoint, data);
        response.EnsureSuccessStatusCode();
        return await response.Content
            .ReadFromJsonAsync<T>();
    }
}
```

8. Литература

- [1] Microsoft, „.NET Multi-platform App UI (.NET MAUI)“, docs.microsoft.com, 2024.
- [2] Microsoft, „ASP.NET Core Web API documentation“, docs.microsoft.com, 2024.
- [3] Npgsql, „Entity Framework Core Provider for PostgreSQL“, npgsql.org, 2024.
- [4] MediatR, „Simple mediator implementation in .NET“, github.com/jbogard/MediatR, 2024.
- [5] Aiven, „Managed Database Services“, aiven.io, 2024.
- [6] Open Food Facts, „Open Food Facts API Documentation“, openfoodfacts.github.io, 2024.
- [7] ZXing.Net.Maui, „Barcode scanning for .NET MAUI“, github.com/redth/ZXing.Net.Maui, 2024.
- [8] Mifflin, M.D. et al., „A new predictive equation for resting energy expenditure in healthy individuals“, The American Journal of Clinical Nutrition, 1990.
- [9] Fowler, M., „Patterns of Enterprise Application Architecture“, Addison-Wesley, 2002.
- [10] Microsoft, „CQRS Pattern“, docs.microsoft.com/en-us/azure/architecture/patterns/cqrs, 2024.

Линк към разработения проект в GitLab:
<https://gitlab.com/MartinElkovGeorgiev/caloriecalc>